Univerza v Ljubljani

Fakulteta za elektrotehniko

David Hožič

RAZVOJ STRATEGIJ UMETNE INTELIGENCE ZA IGRANJE NAMIZNEGA NOGOMETA V PRAVEM OKOLJU

Magistrsko delo
Magistrski študijski program druge stopnje Elektrotehnika

Mentor: doc. dr. Andrej Zdešar
Somentor: asist. dr. Matevž Bošnak

Ljubljana, 2025

UNIVERZA V LJUBLJANI
FE Fakulteta za elektrotehniko

Tržaška cesta 25, p.p. 2999,
1000 Ljubljana, Slovenija
T: 01 476 84 11
dekanat@fe.uni-lj.si
www.fe.uni-lj.si

Št. teme: 00983 / 2025

Datum prijave: 24. 6. 2025

Univerza v Ljubljani, Fakulteta za elektrotehniko izdaja naslednjo nalogo:

Kandidat: **David Hožič**

Naslov: **Razvoj strategij umetne inteligence za igranje namiznega nogometa v pravem okolju**

Development of artificial intelligence policies for playing table football in a real environment

Vrsta naloge: Magistrsko delo
Magistrski študijski program druge stopnje Elektrotehnika

Tematika naloge:
Naloga obravnava razvoj strategij za igranje namiznega nogometa z inteligentnimi agenti, ki so sposobni igrati proti človeku na pravi avtomatizirani igralni mizi. Za izvedbo inteligentne avtonomne obrambe in napada bodo uporabljene metode spodbujevalnega učenja. To zahteva modeliranje igralnega okolja in izvedbo simulatorja, ki omogoča strojno učenje in preizkušanje strategij vodenja igralnih palic. Pri tem se je potrebno soočiti tudi s problemom prenosa regulacijskih algoritmov iz simulacijskega okolja na pravo igralno mizo. Razvite in naučene strategije igranja bodo preizkušene in ovrednotene v simulacijskem in pravem okolju.

Ljubljana, 9. 10. 2025

doc. dr. Andrej Zdešar
Mentor

prof. dr. Igor Škrjanc
Predstojnik katedre

asist. dr. Matevž Bošnak
Somentor

UNIVERZA V LJUBLJANI
Fakulteta
za elektrotehniko
3
LJUBLJANA

prof. dr. Marko Topič
dekan

UNIVERZA V LJUBLJANI
FE Fakulteta za elektrotehniko

Tržaška cesta 25, p.p. 2999,
1000 Ljubljana, Slovenija
T: 01 476 84 11
dekanat@fe.uni-lj.si
www.fe.uni-lj.si

Spodaj podpisani študent, David Hožič, vpisna številka 64200069, avtor pisnega zaključnega dela študija z naslovom: Razvoj strategij umetne inteligence za igranje namiznega nogometa v pravem okolju,

IZJAVLJAM,

1.[1] a) da je pisno zaključno delo študija rezultat mojega samostojnega dela in da sem orodja umetne inteligence uporabljal odgovorno (predvsem s preverjanjem primarnih virov, brez vnašanja avtorsko zaščitenih del v orodja umetne inteligence, s kritičnim vrednotenjem rezultatov, ki so lahko pomanjkljivi, napačni ali neresnični) in sem izključno odgovoren za vsebino avtorskega dela, ki sem ga ustvaril;
   b) da je pisno zaključno delo študija rezultat lastnega dela več kandidatov in izpolnjuje pogoje, ki jih Statut UL določa za skupna zaključna dela študija ter je v zahtevanem deležu rezultat mojega samostojnega dela in da sem orodja umetne inteligence uporabljal odgovorno (predvsem s preverjanjem primarnih virov, brez vnašanja avtorsko zaščitenih del v orodja umetne inteligence, s kritičnim vrednotenjem rezultatov, ki so lahko pomanjkljivi, napačni ali neresnični) in sem izključno odgovoren za vsebino avtorskega dela, ki sem ga ustvaril;
2. da sem pridobil vsa potrebna dovoljenja za uporabo podatkov in avtorskih del v pisnem zaključnem delu študija in jih v pisnem zaključnem delu študija jasno označil;
3. da sem pri pripravi pisnega zaključnega dela študija ravnal v skladu z etičnimi načeli in, kjer je to potrebno, za raziskavo pridobil soglasje etične komisije;
4. da soglašam z uporabo elektronske oblike pisnega zaključnega dela študija za preverjanje podobnosti vsebine z drugimi deli s programsko opremo za preverjanje podobnosti vsebine, ki je povezana s študijskim informacijskim sistemom članice;
5. da na UL neodplačno, neizključno, prostorsko in časovno neomejeno prenašam pravico shranitve avtorskega dela v elektronski obliki, pravico reproduciranja ter pravico dajanja pisnega zaključnega dela študija na voljo javnosti na svetovnem spletu preko Repozitorija UL;
6. da dovoljujem objavo svojih osebnih podatkov, ki so navedeni v pisnem zaključnem delu študija in tej izjavi, skupaj z objavo pisnega zaključnega dela študija.
7. da dovoljujem uporabo mojega rojstnega datuma v zapisu COBISS.

V: Ljubljani
Datum: 10. 10. 2025

Podpis študenta:
Hožič David

[1] Obkrožite varianto a) ali b).

Zahvala

Za pomoč pri izdelavi magistrskega dela se zahvaljujem mentorjema, ki sta mi med izdelavo nudila tako strokovno pomoč kot tudi tehnično podporo. Prav tako se zahvaljujem laboratoriju LAK za razvoj sistema, kjer je bilo delo magistrske naloge preizkušeno. Laboratoriju LTFE se zahvaljujem za dostop do namenske strojne opreme za strojno učenje. Zahvaljujem se tudi laboratoriju LST za kratkotrajen dostop do enako namenske strojne opreme, še v začetku eksperimentalnega dela. Nazadnje se zahvaljujem staršema za večletno podporo.

Povzetek

Delo se ukvarja z razvojem oz. učenjem strategij inteligentnih agentov za igranje namiznega nogometa, ki so sposobni igrati proti človeku na pravi avtomatizirani mizi namiznega nogometa. V delu je bil razvit simulator namiznega nogometa, ki je služil učenju in preizkušanju strategij. Za boljši prenos iz simuliranega okolja v pravo okolje je bila izvedena identifikacija igralnih palic, v okviru katere smo na podlagi posnetkov gibanja pravih palic določili parametre simuliranih palic. Vpliv drugih odstopanj, kot so časovne zakasnitve, smo zmanjšali med razvojem posameznih strategij. Pred razvojem strategij je bil izdelan tudi nasprotnik, ki je bil implementiran v obliki končnega avtomata. Nasprotnik je vključeval vse palice nasprotne ekipe, pri čemer je bila vsaka palica svoj agent. Nasprotnik je služil boljši generalizaciji nekaterih strategij za igro proti človeškim nasprotnikom. S spodbujevalnim učenjem so bile razvite strategije za palico vratarja, obrambno palico in palico napadalca. Strategije so bile vse naučene v simulaciji. Dodatnega učenja v pravem okolju nismo izvedli. Strategije so bile najprej preizkušene v simulaciji, kjer smo ovrednotili sposobnost branjenja lastnega gola (strategiji vratarja in obrambe) oz. sposobnost streljanja na nasprotnikov gol (strategija napadalca). Vse strategije skupaj, z dodatkom palice srednje vrste, ki jo je vodil enak agent kot tisti, ki je vodil nasprotnika, so bile na koncu preizkušene v pravem okolju proti več človeškim parom.

Ključne besede: spodbujevalno učenje, namizni nogomet, simulacija, prenos v pravo okolje, agenti, umetna inteligenca.

Abstract

This thesis explores the development of intelligent agent policies using reinforcement learning, for playing table football against human opponents on a real automated table football table. First, a simulator was developed, whose purpose was both training and testing of policies. For the purposes of simulation-to-real transfer, identification of player rods was performed, where measurements of real rod responses were used to fit simulation parameters. Other discrepancies were compensated during training of individual policy. For better generalization against human opponents, an artificial opponent was developed, which was active during the training process of several policies. The artificial opponent is based on state-machine logic. Three policies were developed using reinforcement learning: a goalkeeper policy, a defense policy and an attacker policy. The midfield rod was made to use the same state-machine logic as the opponent. The developed policies were trained only in simulation, without additional training in the real environment. First, the policies were individually tested in simulation, where we evaluated their ability to defend their own goal (goalkeeper and defense policies) and to score into the opponent's goal (attacker policy). In the end, all policies, with the addition of the midfield state-machine agent, were evaluated against several human opponent pairs in the real environment.

Key words: reinforcement learning, table football, simulation, sim-to-real transfer, agents, artificial intelligence.

Vsebina

1 Uvod **1**

1.1 Povezana dela 2

1.2 Ciljno okolje 3

2 Strojno učenje **5**

2.1 Nadzorovano učenje 5

2.2 Nenadzorovano učenje 6

2.3 Globoko učenje 6

2.3.1 Nevronska mreža 6

2.3.1.1 Polno povezana plast 7

2.3.1.2 Časovno-povratna plast 8

2.3.2 Algoritem učenja 9

3 Spodbujevalno učenje **11**

3.1 Osnovni koncept 11

3.1.1 Agent in okolje 11

3.1.2 Epizoda 12
3.1.3 Nagrada in donos 12
3.1.4 Preslikava stanj v akcije 13
3.2 Primerjava z nadzorovanim učenjem 13
3.3 Raziskovanje in izkoriščanje 13
3.4 Delitev algoritmov 14
3.5 Markovski odločitveni proces 15
3.5.1 Strategija 15
3.5.2 Funkcija verjetnosti prehajanja stanj 15
3.5.3 Vrednosti stanj in akcij 16
3.5.4 Markovska lastnost 17
3.6 Bližinska optimizacija strategije 17
3.6.1 Algoritem 17
3.6.2 Kriterijska funkcija 18
3.6.3 Vrednostna funkcija 19
3.6.4 Vzorčenje akcij 19

4 Metodologija 21

4.1 Razvoj simulatorja 21
4.1.1 Izbira simulacijskega okolja 21
4.1.2 Pretvorba nekonveksne geometrije v konveksno 22

4.1.3 Določitev parametrov žoge 23

4.1.4 Modeliranje palic 23

4.1.4.1 Vodenje 23

4.1.4.2 Identifikacija parametrov 25

4.1.5 Diskretizacija časa 26

4.1.6 Prilagoditve za prenos v pravo okolje 27

4.1.6.1 Zmanjšanje vpliva nepravilnih fizikalnih parametrov 28

4.1.6.2 Modeliranje šuma 28

4.2 Izdelava nasprotnika 29

4.2.1 Stanja 29

4.2.2 Prestrezanje žoge 30

4.2.3 Streljanje žoge 32

4.3 Razvoj strategij s spodbujevalnim učenjem 33

4.3.1 Razvoj strategije napadalca 33

4.3.1.1 Večstopenjsko učenje 33

4.3.1.2 Epizoda 34

4.3.1.3 Prostor stanj 36

4.3.1.4 Prostor akcij 36

4.3.1.5 Nagrada 37

4.3.1.6 Hiperparametri 38

4.3.2 Razvoj strategije vratarja in strategije obrambe 39

4.3.2.1 Epizoda 40

4.3.2.2 Prostor stanj 41

4.3.2.3 Prostor akcij 42

4.3.2.4 Nagrada 42

4.3.2.5 Hiperparametri 43

4.3.3 Zmanjšanje vpliva časovne zakasnitve stanja na delovanje strategij 43

4.3.3.1 Učenje v zakasnjenem okolju 45

4.3.3.2 Odprava vpliva zakasnitve na položaju žoge . . . 47

4.3.3.3 Ocenjevanje hitrosti 48

4.3.3.4 Odprava vpliva zakasnitve na položaju palic . . . 48

5 Rezultati 53

5.1 Končni rezultati 54

5.1.1 Napadalec 54

5.1.2 Vratar in obramba 55

5.1.3 Igra proti človeku 57

5.2 Primerjalne študije 59

5.2.1 Napadalec 60

5.2.1.1 Učenje in vrednotenje z različnimi nasprotniki . . 60

5.2.1.2 Učenje z zahtevano predhodno podajo 61

5.2.2 Vratar in obramba 61

5.2.2.1 Standardizacija stanja 61

5.2.2.2 Dolžnina zlaganja stanj 62

6 Zaključek **65**

A **75**

Seznam slik

1.1 Sistem avtomatiziranega namiznega nogometa FuzbAI 4

2.1 Prikaz nevronske mreže v obliki grafa 7

4.1 Nekonveksni 3D-model in pripadajoča konveksna lupina za zaznavanje trkov 22

4.2 Nekonveksni 3D-model igralca, razbit na dva konveksna dela, in pripadajoča lupina za zaznavanje trkov 23

4.3 Odziv prave in simulirane palice 25

4.4 Končni avtomat FSM-agenta za vodenje iztega 29

4.5 Končni avtomat FSM-agenta za vodenje zasuka 30

4.6 Napoved položaja žoge in trenutni položaj pred prestrežbo ter po prestrežbi 32

4.7 Strel FSM-agenta na gol 32

4.8 Stanje za strategijo vratarja oz. obrambe 42

4.9 Vpliv časovne zakasnitve stanja na uspešnost podaj in zatem strelov na gol 45

4.10 Zlaganje akcij z namenom izpolnitve Markovske lastnosti 46

4.11 Povprečni epizodni donos med učenjem vratarja, katerega stanje vključuje več akcij in tri osnovna stanja ter deluje v okolju z zakasnjenim osnovnim stanjem 46

4.12 Ocenjen položaj žoge pred kompenzacijo in po kompenzaciji . . . 47

4.13 Primer ocenjevanja hitrosti žoge 49

4.14 Arhitektura modela, ki je bil uporabljen za napoved nezakasnjenega položaja palice 50

4.15 Ocenjevanje iztega palice vratarja pri zakasnitvi stanja 40 ms . . . 51

5.1 Podaja in strel RL-agenta napadalca v gol nasprotnika v pravem okolju 55

5.2 Branjenje gola s strani RL-agentov vratarja in obrambe v pravem okolju 57

5.3 Histogram razlik med zadetimi in prejetimi goli pri vrednotenju agentov proti človeškim igralcem 59

5.4 Vpliv standardizacije stanja na povprečni epizodni donos pri učenju strategije vratarja 62

5.5 Povprečni epizodni donos med učenjem strategije vratarja pri različnih dolžinah zlaganja osnovnih stanj 63

A.1 Grafični prikaz potrebnega zamika iztega, relativno na oglišče igralca, za strel v sredino gola 77

Seznam tabel

4.1 Identificirani parametri prostostne stopnje iztega 26

4.2 Identificirani parametri prostostne stopnje zasuka 26

4.3 Vrednosti parametrov za prestrezanje žoge s FSM-agentom 31

4.4 Pogoji za zvišanje stopnje težavnosti pri učenju napadalca 34

4.5 Območja začetnih konfiguracij žoge pri učenju napadalca 35

4.6 Verjetnosti izbire območja konfiguracije za posamezno težavnostno stopnjo 35

4.7 Vrednosti konstantnih hiperparametrov pri učenju napadalca . . . 39

4.8 Prostor za vzorčenje hiperparametrov pri učenju napadalca, ki smo jih optimizirali z algoritmom PBT 40

4.9 Območja vzorčenja vrednosti parametrov začetne konfiguracije žoge pri učenju strategije vratarja oz. strategije obrambe 41

4.10 Vrednosti uteži v funkciji nagrade za strategijo vratarja oz. obrambe 43

4.11 Hiperparametri uporabljeni za učenje strategij vratarja in obrambe 44

4.12 Parametri filtra hitrosti v posameznem koraku glajenja 48

5.1 Delež uspešnih epizod pri streljanju napadalca na gol nasprotnika s strategijo, ki smo jo dodatno učili v zakasnjenem okolju 54

5.2 Delež uspešnih epizod pri streljanju napadalca na gol nasprotnika s strategijo, ki smo jo učili v nezakasnjenem okolju 54

5.3 Delež uspešno ubranjenih strelov vratarja in obrambe s strategijama, ki sta bili dodatno učeni v zakasnjenem okolju 56

5.4 Delež uspešno ubranjenih strelov vratarja in obrambe, s strategijama, ki sta bili učeni v nezakasnjenem okolju 56

5.5 Delež uspešno ubranjenih strelov vratarja in obrambe, strategij. Strategija vratarja je bila učena v nezakasnjenem okolju, medtem ko je bila strategije obrambe dodatno učena v zakasnjenem okolju. 57

5.6 Delež epizod z uspešno podajo in strelom pri zakasnitvi 40 ms, kompenzaciji zakasnitve žoge in lastni kompenzaciji zakasnitve palice 60

5.7 Delež epizod z uspešno zadetim golom pri zakasnitvi 40 ms, kompenzaciji zakasnitve položaja žoge in lastni kompenzaciji zakasnitve položaja palice 61

Seznam uporabljenih simbolov

V delu so uporabljene sledeče splošne veličine in simboli:

Ime	Simbol
čas	t
(časovni) korak	k
frekvenca	f
vzorčni čas	T_s
hitrost	v
pospešek	a
izteg	e
zasuk	r
položaj	p
sila	f
stanje	s
akcija	a
meritev stanja	o
funkcija nagrade	R
donos (nagrade)	G
faktor popusta (nagrade)	γ
strategija	π

Manj splošne veličine oz. simboli so razloženi ob prvi uporabi. Prav tako nekatere veličine nimajo enot, logičnega pomena ali pa so enote odvisne od konteksta.

Vektorji in matrike so zapisani odebeljeno.

1 Uvod

V zadnjih letih je mogoče zaznati precejšen porast zanimanja za umetno inteligenco (angl. *artificial intelligence*). Večina ljudi ob tej besedni zvezi pomisli na velike jezikovne modele (angl. *Large Language Models* – LLM), kamor spada tudi ChatGPT [1], v resnici pa umetna inteligenca predstavlja precej širše področje. V splošnem je umetna inteligenca področje, ki se ukvarja z gradnjo avtomatiziranih sistemov, sposobnih reševanja problemov, ki tipično zahtevajo intelektualne sposobnosti človeka [2]. Sama definicija umetne inteligence ne definira njene izvedbe, zato pod njo spadajo tudi ekspertni sistemi, ki so v splošnem izvedeni z eksplicitnimi pravili, z namenom posnemanja človeškega strokovnjaka na določenem področju [3]. Danes se v ospredju umetne inteligence bolj pogosto od ekspertnih sistemov pojavljajo sistemi učeni s strojnim učenjem. V nadaljevanju opišemo strojno učenje s poudarkom na spodbujevalnem učenju (angl. *reinforcement learning* – RL), ki je ključnega pomena za umetno inteligenco v fizičnem svetu.

V tem delu se osredotočamo na razvoj agentov, oziroma njihovih strategij, za igranje namiznega nogometa na avtomatizirani mizi namiznega nogometa z uporabo algoritmov spodbujevalnega učenja. Cilj so strategije za različne igralne palice, ki bodo sposobne braniti lasten gol in streljati na gol nasprotnika. Razvite strategije morajo delovati v pravem okolju, zato je potrebno pri delu upoštevati šum in zakasnitve.

Struktura dela po poglavjih je sledeča:

- Strojno učenje: predstavimo osnovne pojme strojnega učenja in opišemo osnove globokega učenja;

- Spodbujevalno učenje: podrobneje opišemo spodbujevalno učenje in algoritem spodbujevalnega učenja, ki smo ga v tem delu uporabili za izdelavo strategij;
- Metodologija: opišemo razvoj simulatorja, razvoj strategij v simulatorju in uporabljene metode za prenos v pravo okolje;
- Rezultati: podamo rezultate vrednotenja končnih modelov v simulatorju proti umetnemu nasprotniku in v pravem okolju proti človeškim parom. Predstavimo tudi rezultate primerjalnih študij.

1.1 Povezana dela

Kljub temu, da so avtomatizirane mize za igranje namiznega nogometa redke, obstaja nekaj povezanih del z razvojem agentov za igranje namiznega nogometa. Tu so omenjena dela, kjer so avtorji agente oz. njihove strategije izdelali s spodbujevalnim učenjem.

V [4] so avtorji razvili avtomatiziran sistem za igranje namiznega nogometa, zatem pa s spodbujevalnim učenjem razvili agenta napadalca, ki so ga učili streljanja žoge na gol. Pri tem je bila žoga v okolje postavljena naključno v bližino napadalca in imela ničelno hitrost. Agent je bil učen v simulaciji in preizkušen tako v simulaciji kot tudi na pravi mizi.

V [5] so avtorji s spodbujevalnim učenjem razvili agenta vratarja. Cilj slednjega je branjenje strelov na gol s premiki levo / desno / nikamor. Agent je bil učen v simulaciji in preizkušen tako v simulaciji kot tudi na pravi mizi.

V [6] so avtorji razvili dva agenta: vratarja in napadalca. Agenta sta bila učena drug proti drugemu ob istem času. Delovanje agentov je bilo preizkušeno le v simulaciji.

V [7] so avtorji razvili več obnašanj agentov. Obnašanja vključujejo podajanje in preigravanje žoge med igralci na isti palici in podajanje žoge drugi palici mimo nasprotnikove palice. Delo je bilo izvedeno le v simulaciji.

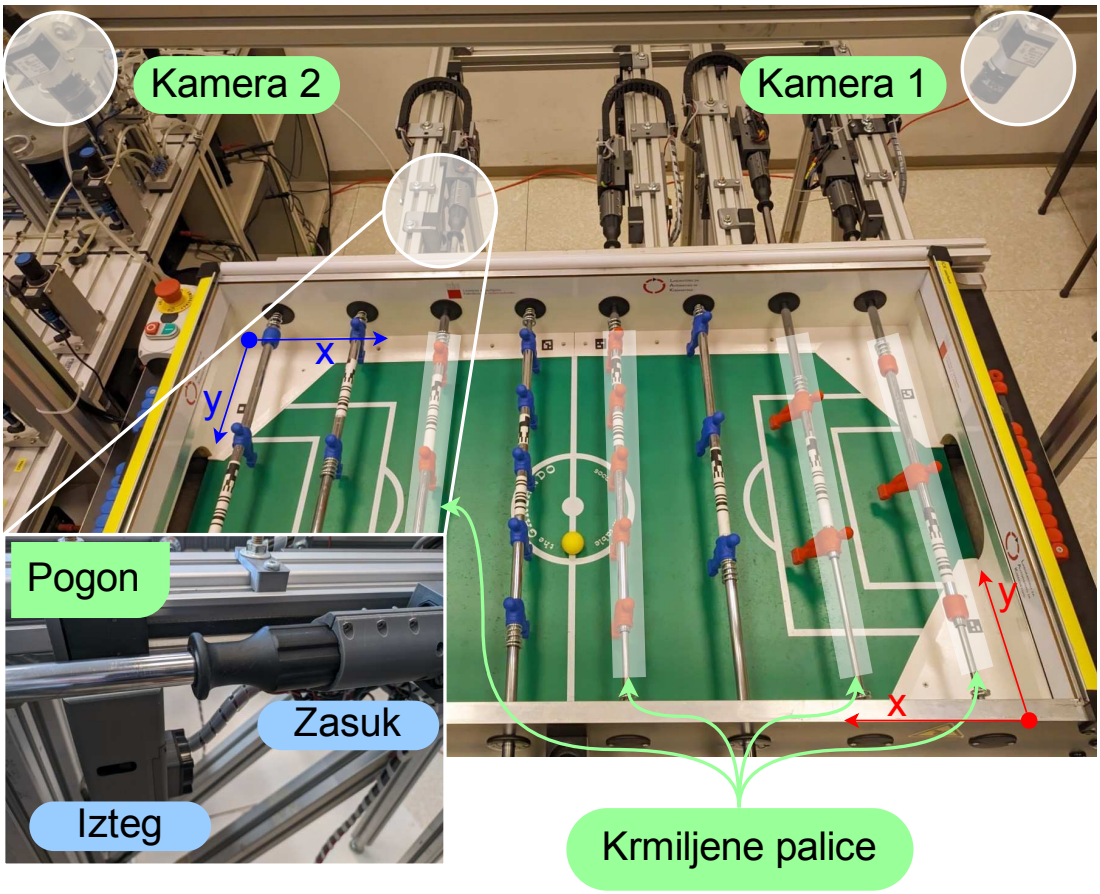
Naše delo se od prej omenjenih razlikuje predvsem v tem, da smo v igri po-

skušali združiti vse palice (tri izdelane z učenjem in eno s klasičnimi metodami). Dodatno smo poskusili izboljšati prenos iz simulacije v pravo okolje z upoštevanjem in kompenzacijo časovnih zakasnitev, ki so del vsakega sistema v pravem svetu. V času pisanja nismo našli nobenega predhodnega dela, ki bi v okolju avtomatiziranega namiznega nogometa upoštevalo zakasnitve.

1.2 Ciljno okolje

Ciljno okolje, tj. sistem avtomatiziranega namiznega nogometa FuzbAI, je prikazano na sliki 1.1. Sistem je v osnovi miza za igranje namiznega nogometa, izdelana v podjetju Garlando. Za namen ocenjevanja položaja žoge in palic sta nad mizo pritrjeni dve kameri, ki imata vzorčno frekvenco $f_\mathrm{s} = 100\,\mathrm{Hz}$. Celoten sistem zaznavanja in merjenja leg palic je podrobneje opisan v [8]. Ena stran mize je namenjena človeški ekipi, na nasprotni strani pa sta na vsako palico pritrjena dva motorja: eden za prostostno stopnjo iztega, drugi za prostostno stopnjo zasuka. Motorja je mogoče krmiliti na daljavo preko protokola HTTP (angl. *HyperText Transfer Protocol*). Za namen notranjega vodenja imata motorja vsak svoj dajalnik (angl. *encoder*), ki ga lahko uporabimo za merjenje položaja brez zanašanja na kameri.

V nadaljevanju je opisan razvoj agentov oz. njihovih strategij za igranje v ciljnem okolju, še prej pa je podano teoretično ozadje, ki je potrebno za razumevanje razvoja.



Slika 1.1: Sistem avtomatiziranega namiznega nogometa FuzbAI

2 Strojno učenje

Strojno učenje predstavlja področje, ki se ukvarja z učenjem iz podatkov. V splošnem metode strojnega učenja lahko razdelimo na nadzorovano učenje (angl. *supervised learning*), nenadzorovano učenje (angl. *unsupervised learning*) in spodbujevalno učenje (angl. *reinforcement learning*).

Med neodvisno podskupino strojnega učenja spada tudi globoko učenje, ki ga zaznamuje uporaba globokih nevronskih mrež, vključuje pa tudi vse tri zgoraj omenjene delitve.

2.1 Nadzorovano učenje

Nadzorovano učenje predstavlja nabor algoritmov za izdelavo modelov oz. funkcijskih aproksimatorjev, ki na podlagi vrednosti vhodnih spremenljivk oz. značilk (angl. *features*) napovejo vrednost izhodne spremenljivke oz. oznake (angl. *label*). Da te algoritme lahko uporabimo, potrebujemo nabor podatkov (angl. *dataset*), kjer je znano, katera spremenljivka je izhodna spremenljivka [9].

Nadzorovano učenje lahko dodatno razdelimo še na podlagi vrste oznake, ki je lahko kvantitativna ali kvalitativna. V primeru kvantitativne narave oznake govorimo o regresiji (npr. napoved cene hiše na podlagi njenih lastnosti), v primeru kvalitativne pa o klasifikaciji (npr. napoved vrste živali na sliki).

Med klasične algoritme spadajo: linearna regresija, logistična regresija, odločitvena drevesa ipd.

2.2 Nenadzorovano učenje

Nenadzorovano učenje predstavlja nabor algoritmov, ki operirajo na podatkih brez določenega izhoda. Tu ne moremo napovedati nekega izhoda, saj le tega ni, lahko pa na primer iščemo povezave med ostalimi spremenljivkami. Klasičen primer iskanja povezav med spremenljivkami je gručenje (angl. *clustering*), kjer točke na podlagi njihove oddaljenosti do drugih točk razporedimo v gruče. Iz gruč lahko na primer izluščimo stranke, ki spletni trgovini prinesejo večje zaslužke. To je uporabno za ugotavljanje lastnosti strank, ki vodijo do večjega zapravljanja in tako do večjih zaslužkov [9].

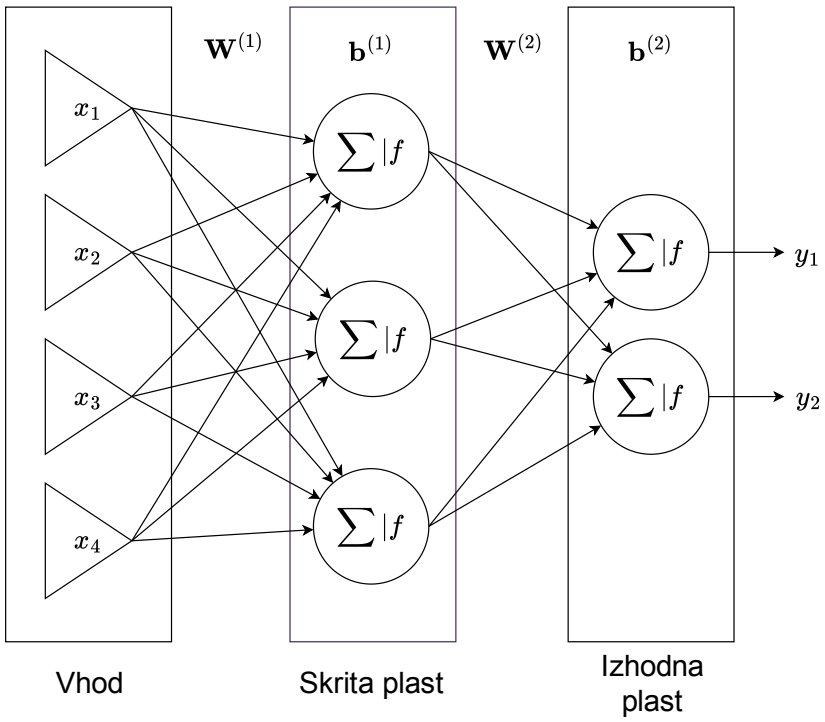
Med klasične algoritme spadajo: gručenje k-tih povprečij (angl. *k-means clustering*), metoda analize glavnih komponent (angl. *Principal Component Analysis* – PCA), avtokodirniki (angl. *autoencoders*) ipd.

2.3 Globoko učenje

Globoko učenje (angl. *deep learning*) predstavlja moderno strojno učenje in vključuje rabo (globokih) nevronskih mrež. Te predstavljajo univerzalne aproksimatorje, sposobne aproksimiranja poljubno kompleksnih funkcij, z različnim številom in tipi vhodov kot tudi izhodov. Aplikacije globokega učenja so precej široke, vključujejo pa računalniški vid, prepoznavo in sintezo besedila ter govora, spodbujevalno učenje in druga področja.

2.3.1 Nevronska mreža

V splošnem lahko nevronsko mrežo predstavimo kot graf z več plastmi, ki vsebuje vozlišča (angl. *nodes*) in robove (angl. *edges*) kot prikazuje slika 2.1. Oznaka x_n predstavlja n-to vhodno spremenljivko, $\mathbf{W}^{(m)}$ matriko uteži v m-ti plasti, $\mathbf{b}^{(m)}$ vektor začetnih vrednosti v m-ti plasti, y_n n-to izhodno spremenljivko in f poljubno aktivacijsko funkcijo, ki se v splošnem lahko razlikuje od vozlišča do vozlišča, pogosto pa se razlikuje le med plastmi.



Slika 2.1: Prikaz nevronske mreže v obliki grafa

Nevronske mreže so običajno sestavljene iz več plasti, ki se razlikujejo tako po številu vozlišč kot tudi vrsti. Če nevronska mreža vsebuje več skritih plasti, jo imenujemo globoka nevronska mreža. V nadaljevanju sta opisani dve vrsti plasti iz velikega nabora vrst, ki se pogosto pojavita v spodbujevalnem učenju in smo jih v tem delu uporabljali tudi sami.

2.3.1.1 Polno povezana plast

Polno povezana plast (angl. *fully connected layer*) predstavlja osnovni gradnik nevronskih mrež. Pravzaprav je ta prikazana na sliki 2.1 (skrita in izhodna plast). Zanjo je značilno, da je vsako vozlišče (nevron) v plasti povezano z vsakim vozliščem iz prejšnje plasti. Nevronski mreži, sestavljeni le iz teh plasti, pravimo večplastni perceptron (angl. *Multi-Layer Perceptron* – MLP) [10].

Matematično lahko dvoplastni MLP na sliki 2.1 zapišemo kot:

$$\begin{aligned} \mathbf{z}(\mathbf{x}) &= f(\mathbf{b}^{(1)} + \mathbf{W}^{(1)}\mathbf{x}) \\ \mathbf{y}(\mathbf{z}(\mathbf{x})) &= f(\mathbf{b}^{(2)} + \mathbf{W}^{(2)}\mathbf{z}(\mathbf{x})), \end{aligned} \tag{2.1}$$

kjer $\mathbf{z}$ predstavlja izhod prve plasti, $\mathbf{y}$ izhod druge plasti, $\mathbf{x}$ vhod, f (nelinearno) zvezno odvedljivo aktivacijsko funkcijo, $\mathbf{b}^{(m)}$ vektor začetne vrednosti (angl. *bias*) v m-ti plasti ter $\mathbf{W}^{(m)}$ matriko uteži v m-ti plasti, dimenzij $v \times s$. Oznaka v predstavlja število vozlišč v plasti, s pa število vhodov v plast.

Aktivacijske funkcije so matematične funkcije, ki vpeljejo nelinearnost za namene modeliranja nelinearnih funkcij. Pogoste aktivacijske funkcije vključujejo ReLU (angl. *Rectified Linear Unit*), definirano kot $f(x) = \max(x, 0)$; hiperbolični tangens, ki ga zapišemo kot $f(x) = \tanh(x)$; sigmoidno funkcijo, ki jo definiramo kot $f(x) = \frac{1}{1+e^{-x}}$ in druge.

2.3.1.2 Časovno-povratna plast

Časovno-povratne plasti (angl. *recurrent layers*) omogočajo napovedovanje na podlagi zgodovine vhoda. To dosežejo s pomnjenjem skritega stanja, ki ga lahko definiramo kot:

$$\mathbf{h}(k) = f(\mathbf{A}\mathbf{h}(k-1) + \mathbf{B}\mathbf{x}(k)), \tag{2.2}$$

kjer $\mathbf{h}$ predstavlja skrito stanje, $\mathbf{x}$ vhod, k (časovni) korak, f (nelinearno) zvezno odvedljivo funkcijo ter $\mathbf{A}$ in $\mathbf{B}$ matriki uteži [11]. Skrito stanje je tudi izhod časovno-povratne plasti.

Časovno-povratna plast definirana z enačbo (2.2) predstavlja šolski primer in v praksi ni pogosto uporabljena zaradi problemov z izginjanjem in prekomerno rastjo gradientov. Problem rešujeta vrsti plasti LSTM (angl. *Long Short-Term Memory*) [12] in GRU (angl. *Gated Recurrent Unit*) [13].

Ker morajo časovno-povratne plasti vhode procesirati zaporedno, so pri učenju počasnejše od polno povezanih plasti. Moderno alternativo, ki rešuje ta problem, predstavlja transformerska arhitektura [14], ki podatkov med učenjem ne procesira zaporedno, temveč vzporedno.

2.3.2 Algoritem učenja

Nevronske mreže je potrebno učiti oz. optimizirati. Medtem ko se algoritmi učenja oz. optimizacije klasičnih metod precej razlikujejo med seboj, se nevronske mreže najpogosteje uči z algoritmi, ki temeljijo na algoritmu gradientnega spusta (angl. *gradient descent*), definiranega kot:

$$w_k \leftarrow w_k - \alpha \frac{\partial E(\mathbf{W})}{\partial w_k}, \tag{2.3}$$

kjer w_k predstavlja k-to utež oz. koeficient matrike uteži, α hiperparameter hitrosti učenja (angl. *learning rate*) in $E(\mathbf{W})$ funkcijo, odvisno od vseh uteži, ki jo želimo minimizirati.

Algoritem gradientnega spusta zahteva izračun parcialnega odvoda funkcije $E(\mathbf{W})$ po posamezni uteži. Postopek izračuna parcialnih odvodov po utežeh se imenuje vzvratna propagacija (angl. *backpropagation*) [15]. Sodobne knjižnice globokega učenja, kot je PyTorch [16], vzvratno propagacijo izvajajo avtomatično.

V praksi se osnovnega algoritma gradientnega spusta ne uporablja, temveč se uporablja njegove izboljšave (npr. Adam [17]).

3 Spodbujevalno učenje

Spodbujevalno učenje predstavlja področje, ki se ukvarja z reševanjem problemov na podlagi poskušanja in učenja iz napak. Je podzvrst strojnega učenja, ki je bilo v samem začetku precej neodvisno področje od nadzorovanega in nenadzorovanega učenja, danes pa je ta meja zamegljena.

3.1 Osnovni koncept

3.1.1 Agent in okolje

Ospredje spodbujevalnega učenja je agent, ki deluje v okolju. Agent je sposoben zaznavati stanje okolja in na podlagi tega izvajati akcije, ki vplivajo na agenta samega in/ali na okolje. Agent po akciji prejme tudi nagrado (ali kazen), ki agenta spodbudi (ali odvrne) od izvajanja iste akcije v določenem stanju [18]. Cilj spodbujevalnega učenja je najti preslikavo stanja na akcijo, ki bo vodila do optimalne rešitve oz. do najvišje akumulirane nagrade.

Na primer, agent je lahko človek, ki krmili kolo in se pelje po gozdu. Akcije usmerjajo kolo levo, desno ali naravnost na podlagi stanja gozdne poti, ki jo agent vidi z očmi. Gozdne poti so polne lukenj in neravnih odsekov, ki se jim mora agent izogniti. Če agent zapelje v luknjo, bo padel in občutil bolečino, kar predstavlja kazen (negativno nagrado). Ob ponovnem poskusu vožnje se bo agent luknjam izognil že daleč pred njimi, da se izogne poškodbi oz. bolečini. Zaznavanje stanja, izvajanje akcij in prejemanje nagrade se ponavlja dokler agent ne zapusti gozda.

3.1.2 Epizoda

Pomemben koncept v okviru spodbujevalnega učenja je epizoda. Ta je definirana kot zaporedje stanj, akcij in nagrad od nekega začetnega stanja, vse do končnega stanja, neveljavnega stanja ali pa na primer preteka omejitve časa oz. korakov. Slednjega pravzaprav ne štejemo kot konec (angl. *termination*) epizode, temveč kot njeno okrnitev (angl. *truncation*).

3.1.3 Nagrada in donos

Algoritmi spodbujevalnega učenja počnejo enako kot človek v primeru vožnje kolesa po gozdu. To pomeni, da iščejo tiste akcije, ki vodijo do najvišje možne pričakovane akumulirane nagrade [19]. Akumulirani nagradi v prihodnosti rečemo donos (angl. *return*), ki ga lahko definiramo kot:

$$G_k = \sum_{\tau=0}^{K-1} R(\mathbf{s}_{k+\tau}, \mathbf{a}_{k+\tau}), \tag{3.1}$$

kjer G predstavlja donos, R nagrado, $\mathbf{a}$ akcijo, k časovni korak, $\mathbf{s}$ stanje in K preostalo število korakov do konca epizode. Alternativo enačbi (3.1) predstavlja enačba (3.2), ki nagradam v bližnji prihodnosti da več teže kot bolj oddaljenim.

$$G_k^{\gamma} = \sum_{\tau=0}^{K-1} \gamma^{\tau} R(\mathbf{s}_{k+\tau}, \mathbf{a}_{k+\tau}) \tag{3.2}$$

V enačbi (3.2) $\gamma \in [0,\ 1]$ predstavlja faktor popusta (angl. *discount factor*) nagrade. S tem je enačba (3.2) v splošnem bolj primerna za učenje od enačbe (3.1), saj (v primeru $\gamma < 1$) omogoča neskončno dolge epizode [18].

Enačbi (3.1) in (3.2) predstavljata donos, če epizodo začnemo v stanju $\mathbf{s}_k$ in izvedemo akcijo $\mathbf{a}_k$. Z vidika epizode je smiselno zapisati epizodni donos:

$$G_{\text{ep}} = \sum_{k=0}^{K-1} R_k, \tag{3.3}$$

kjer $k = 0$ predstavlja začetek epizode.

3.1.4 Preslikava stanj v akcije

Da agent na podlagi zaznanega stanja lahko izvede akcijo, potrebuje neko obliko preslikave iz stanja v akcijo. V okviru spodbujevalnega učenja tej preslikavi pravimo strategija (angl. *policy*), pri čemer oblika strategije ni določena vnaprej. Strategija je lahko določena na različne načine: kot tabela, funkcijski aproksimator (npr. nevronska mreža) itd.

3.2 Primerjava z nadzorovanim učenjem

Kot omenjeno, se spodbujevalno učenje ukvarja z optimizacijo akcij. To ne pomeni, da akcij ne moremo optimizirati z nadzorovanim učenjem. Če imamo podatke o referenčnih trajektorijah $(\mathbf{s}, \mathbf{a})$, lahko izdelamo model, ki minimizira napako med napovedano akcijo in akcijo iz referenčnih podatkov. Takšen primer podaja [20], kjer so bili posnetki slik in akcij igrice Minecraft uporabljeni za učenje igranja le-te.

Spodbujevalno učenje se od nadzorovanega razlikuje predvsem v tem, da mora agent sam ugotoviti katere akcije so najboljše za rešitev problema. To lahko vodi do zanimivih rešitev, vključno s tistimi, ki jih ne pričakujemo.

3.3 Raziskovanje in izkoriščanje

Za določitev optimalne akcije sta potrebna dva načina delovanja: raziskovanje (angl. *exploration*) in izkoriščanje (angl. *exploitation*). Izkoriščevalno agent deluje, ko izbira akcijo, ki je ocenjena kot najboljša, v nasprotnem primeru pa deluje raziskovalno [19]. Med tema načinoma mora obstajati neko ravnovesje. Le raziskovalno (naključno) agent ne more delovati, saj tako problem ne bo nikoli rešen. Prav tako ne more delovati le izkoriščevalno, saj brez raziskovanja ni razvidno katera akcija je najboljša.

3.4 Delitev algoritmov

Algoritme spodbujevalnega učenja je mogoče razdeliti na različne načine, pri čemer se posamezne delitve med seboj ne izključujejo. Algoritme lahko na primer razdelimo glede na:

- uporabo modela okolja:
  - ekspliciten model okolja (angl. *model-based*);
  - vzorčenje okolja (angl. *model-free*);
- način učenja iz okolja:
  - neposredni algoritmi: učenje iz podatkov, ki so bili pridobljeni s trenutno strategijo (angl. *on-policy*);
  - posredni algoritmi: učenje na podlagi podatkov, ki so bili pridobljeni z drugimi strategijami (angl. *off-policy*);
- način pridobitve podatkov:
  - iz okolja sproti med učenjem (angl. *online*);
  - iz obstoječih podatkov (angl. *offline*);
- obliko strategije:
  - implicitna preslikava vrednostne funkcije (v danem stanju izberemo akcijo z najvišjo vrednostjo vrednostne funkcije);
  - eksplicitna preslikava;
- in drugo.

Primeri različnih metod vključujejo:

- Q-učenje [21] in njegovo sodobno različico DQN (angl. *Deep Q Network*) [22], ki sta posredna algoritma in za izbiro akcij uporabljata vrednostno funkcijo;
- SARSA (angl. *State-Action-Reward-State-Action*) [23], ki je neposreden algoritem in prav tako temelji na vrednostni funkciji;
- REINFORCE [24], ki neposredno optimizira strategijo preko gradienta strategije brez uporabe vrednostne funkcije, oziroma slednja ni potrebna;
- SAC (angl. Soft Actor-Critic) [25], ki strategijo optimizira z uporabo gradienta strategije na podlagi ocene vrednostne funkcije. Pristop optimizacije eksplicitne strategije na podlagi vrednostne funkcije imenujemo igralec-

ocenjevalec (angl. *actor-critic*) [26], kjer strategija igra vlogo igralca, vrednostna funkcija pa vlogo ocenjevalca.

3.5 Markovski odločitveni proces

Interakcija agenta z okoljem v poglavju 3.1.1 in opis donosa v poglavju 3.1.3 sovpadata z Markovskim odločitvenim procesom (angl. *Markov Decision Process* – MDP), ki predstavlja matematično definicijo spodbujevalnega učenja.

Markovski odločitveni proces je definiran s prostorom akcij $\mathcal{A}$, prostorom stanj $\mathcal{S}$, funkcijo verjetnosti prehajanja stanj $p(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$ in uteženim donosom G^γ. V nadaljevanju je predstavljena osnovna teorija MDP, pri čemer pretežno sledimo definicijam v [19].

3.5.1 Strategija

V poglavju 3.1.4 smo strategijo definirali kot preslikavo iz stanja na akcijo. Tej definiciji sledi tudi MDP, z razliko, da je tu preslikava stohastična. Zapišemo jo lahko kot $\pi(\mathbf{a}|\mathbf{s})$, kjer π predstavlja strategijo in pomeni verjetnost za izbiro akcije $\mathbf{a}$, ko smo v danem stanju $\mathbf{s}$.

3.5.2 Funkcija verjetnosti prehajanja stanj

Klasičen opis spodbujevalnega učenja predpostavlja, da agent deluje v nekem okolju. V zapisu MDP je okolje predstavljeno s funkcijo verjetnosti prehajanja stanj $p(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$. Ta podaja verjetnost, da ob izvedbi akcije $\mathbf{a}_k$ v stanju $\mathbf{s}_k$ agent preide v novo stanje $\mathbf{s}_{k+1}$. Če bi poznali funkcijo $p(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$, nam akcij v okolju ne bi bilo potrebno izvajati, saj so vse informacije o posledicah na stanje na voljo že s funkcijo $p(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$.

V praksi funkcije $p(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$ ne poznamo. Ker je potrebna za izračun vrednosti stanj oz. parov stanje-akcija, se jo v praksi ocenjuje implicitno, in sicer z metodo Monte Carlo.

3.5.3 Vrednosti stanj in akcij

Pogoj za delovanje algoritmov spodbujevalnega učenja je zmožnost vrednotenja stanj in akcij. V poglavju 3.1.3 smo omenili, da za učenje potrebujemo izračun donosa, ki je v splošnem stohastičen. Izračunati moramo torej matematično pričakovanje donosa. V zapisu MDP je to vrednost para stanje-akcija oz. Q-funkcija:

$$Q_\pi(\mathbf{s}, \mathbf{a}) = \mathbb{E}_\pi[G_k^\gamma | \mathbf{s}_k = \mathbf{s}, \mathbf{a}_k = \mathbf{a}], \tag{3.4}$$

kjer Q_π označuje vrednost para stanje-akcija ob delovanju strategije π, $\mathbf{a}$ akcijo izvedeno v stanju $\mathbf{s}$ in G_k^γ donos, ki je definiran z enačbo (3.2). $\mathbb{E}_\pi$ združuje matematično pričakovanje zaradi stohastične funkcije prehajanja stanj $p(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k)$ in matematično pričakovanje zaradi stohastične strategije π. V primeru, da z učenjem določimo optimalno strategijo π^*, lahko zapišemo njeno optimalno funkcijo para stanje-akcija kot $Q^*(\mathbf{s}, \mathbf{a})$. Akcije, ki vodijo do najboljšega delovanja, so v tem primeru tiste, ki dajo največjo vrednost $Q^*(\mathbf{s}, \mathbf{a})$:

$$\mathbf{a} = \operatorname*{argmax}_{\mathbf{a}'} Q^*(\mathbf{s}, \mathbf{a}'). \tag{3.5}$$

Ker akcije izbira strategija π, lahko zapišemo še vrednost samega stanja oz. vrednostno funkcijo (angl. *value function*):

$$V_\pi(\mathbf{s}) = \mathbb{E}_{\mathbf{a} \sim \pi(\cdot|\mathbf{s})}[Q_\pi(\mathbf{s}, \mathbf{a})]. \tag{3.6}$$

Vrednostna funkcija iz enačbe (3.6) je pogosto uporabljena v gradientnih metodah.

Nekateri algoritmi namesto enačb (3.4) in (3.6) uporabljajo t. i. prednostno funkcijo (angl. *advantage function*):

$$A_\pi(\mathbf{s}, \mathbf{a}) = Q_\pi(\mathbf{s}, \mathbf{a}) - V_\pi(\mathbf{s}). \tag{3.7}$$

Prednostna funkcija iz enačbe (3.7) podaja informacijo o tem koliko je neka akcija boljša od povprečne akcije v danem stanju, pripomore pa k zmanjšanju variance gradientov [27].

3.5.4 Markovska lastnost

Pogoj za modeliranje problema kot MDP je izpolnjena Markovska lastnost. Stanje izpolnjuje Markovsko lastnost, če je odvisno le od prejšnjega stanja in izbrane akcije v tem stanju, brez odvisnosti od preteklosti. Funkcijo verjetnosti prehajanja stanj, ki izpolnjujejo Markovsko lastnost, lahko zapišemo kot:

$$p(\mathbf{s}_{k+1}|\mathbf{s}_k, \mathbf{a}_k) = p(\mathbf{s}_{k+1}|\mathbf{s}_{0:k}, \mathbf{a}_{0:k}), \tag{3.8}$$

kjer oznaka $0{:}k$ označuje nabor vrednosti od vključno koraka 0 do vključno koraka k.

3.6 Bližinska optimizacija strategije

V tem poglavju je opisan algoritem Bližinske optimizacije strategije (angl. *Proximal Policy Optimization* – PPO), ki smo ga v našem delu pretežno uporabljali. Opis je povzet po uradnem članku [28].

3.6.1 Algoritem

Algoritem PPO spada med gradientne metode. Uradno velja za neposreden algoritem, saj pred vsakim korakom učenja iz okolja pridobi nove vzorce s trenutno strategijo. Kljub temu, zaradi več iteracij optimizacijskega koraka z istimi podatki, lahko rečemo, da je delno posreden. Ker pri optimizaciji strategije uporablja vrednostno funkcijo, velja za algoritem tipa igralec-ocenjevalec.

Delovanje PPO je predstavljeno z algoritmom 1, kjer $\mathcal{D}$ predstavlja pomnilnik (angl. *rollout buffer*) za hrambo vzorcev oblike $(\mathbf{s}_k, \mathbf{a}_k, R_k, d_k)$, $\mathbf{s}$ stanje, $\mathbf{a}$ akcijo, R nagrado, $d \in [0,\ 1]$ oznako ali je epizoda zaključena, K število vzorčenih časovnih korakov pred učenjem in M število korakov optimizacije. V primeru nezaključene epizode ob zadnjem koraku ($d = 0$) se donos od tega koraka naprej izračuna iz ocene vrednostne funkcije, s postopkom začetnega ocenjevanja (angl. *bootstrapping*) vrednosti [19].

Algoritem 1 Bližinska optimizacija strategije

Vhod: parametri strategije θ in parametri vrednostne funkcije ψ.

1: **for** $i = 1, 2, \ldots$ **do** ▷ iteracija algoritma PPO
2: $\mathcal{D} \leftarrow \emptyset$
3: **for** $k = 0, 1, \ldots, K-1$ **do** ▷ korak v okolju
4: Vzemi vzorec oblike $(\mathbf{s}_k, \mathbf{a}_k, R_k, d_k)$ iz okolja in jih grupirane po epizodah shrani v $\mathcal{D}$.
5: **end for**
6: Z algoritmom GAE izračunaj ocene prednosti $\hat{A}(\mathbf{s}, \mathbf{a})$ za vzorce v $\mathcal{D}$ na podlagi vrednostne funkcije V_ψ in jih dodaj v $\mathcal{D}$.
7: $\theta' \leftarrow \theta$
8: **for** $m = 1, \ldots, M$ **do** ▷ optimizacijski korak
9: Posodobi parametre strategije z gradientnim dvigom na $J(\theta', \theta)$ in parametre vrednostne funkcije z gradientnim spustom na $J(\psi)$, na podlagi vzorcev v $\mathcal{D}$.
10: **end for**
11: $\theta \leftarrow \theta'$
12: **end for**

3.6.2 Kriterijska funkcija

Kriterijsko funkcijo $J(\theta', \theta)$ lahko v splošni obliki zapišemo z enačbo (3.9), kjer $\pi_{\theta'}$ predstavlja trenutno strategijo, π_θ strategijo pred prvim optimizacijskim korakom, $A(\mathbf{s}, \mathbf{a})$ funkcijo prednosti, $\mathcal{H}$ entropijo in KL Kullback-Leiblerjevo divergenco [29]. Slednja predstavlja mero odstopanja $\pi_{\theta'}$ od π_θ.

$$J(\theta', \theta) = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{s},\mathbf{a}) \in \mathcal{D}} \begin{pmatrix} \min\left(j\, \hat{A}(\mathbf{s}, \mathbf{a}), \mathrm{clamp}(j,\ 1-\epsilon,\ 1+\epsilon)\, \hat{A}(\mathbf{s}, \mathbf{a})\right) \\ +\alpha\, \mathcal{H}(\pi_{\theta'}(\cdot|\mathbf{s})) - \beta\, \mathrm{KL}(\pi_\theta(\cdot|\mathbf{s}) \mid\mid \pi_{\theta'}(\cdot|\mathbf{s})) \end{pmatrix}, \tag{3.9}$$

$$\text{kjer je} \quad j = \frac{\pi_{\theta'}(\mathbf{a}|\mathbf{s})}{\pi_\theta(\mathbf{a}|\mathbf{s})}, \quad \mathrm{clamp}(j, a, b) = \min(\max(j, a), b) \quad \text{in} \quad \epsilon > 0.$$

V [28] KL divergenco podajo kot alternativo omejevanju razmerja j s funkcijo clamp. Kriterijska funkcija v enačbi (3.9) vsebuje obe (KL divergenco in uporabo funkcije clamp), saj je algoritem PPO tako implementiran v knjižnici, ki smo jo uporabljali pri našem delu. Ta knjižnica je Ray RLlib [30].

3.6.3 Vrednostna funkcija

Za uporabo enačbe (3.9) predhodno potrebujemo oceno funkcije prednosti $\hat{A}$, za katero, na podlagi enačbe (3.7), v splošnem potrebujemo tako vrednost para stanje-akcija, kot tudi vrednost stanja. Določeni algoritmi, vključno z algoritmom PPO, za izračun (ocene) prednosti izrabljajo posplošeni izračun prednosti (angl. *Generalized Advantage Estimation* – GAE) [27], ki za ocenjevanje prednosti potrebuje le vrednostno funkcijo. Slednjo v splošnem lahko pridobimo na več načinov. Eden izmed njih je minimizacija povprečne kvadratne napake (angl. *Mean Squared Error* – MSE):

$$J(\psi) = \frac{1}{N_\tau L} \sum_{\tau \in \mathcal{D}} \sum_{l=0}^{L} (V_\psi(\mathbf{s}_l) - G_l^\gamma)^2, \tag{3.10}$$

ki smo jo prilagodili po [31]. V enačbi (3.10) G_l^γ pomeni preostali donos od časovnega koraka l naprej, izračunan po enačbi (3.2), V_ψ vrednostno funkcijo, $\mathcal{D}$ pomnilnik iz algoritma 1, L dolžino trajektorije, τ trajektorijo in N_τ število epizodnih trajektorij v $\mathcal{D}$.

3.6.4 Vzorčenje akcij

Smiselno je omeniti tudi razlike med strategijo v primeru diskretnih in zveznih akcij. V primeru diskretnih akcij nevronska mreža, ki predstavlja strategijo, na izhodu neposredno podaja verjetnosti za posamezne akcije. Izhodov je toliko kot je različnih akcij. V primeru determinističnega delovanja ($\pi(\mathbf{s}) \rightarrow \mathbf{a}$) izberemo akcijo z najvišjo (napovedano) verjetnostjo.

V primeru zveznih akcij so stvari bolj zapletene, saj je na voljo neskončno mnogo akcij. Izhod strategije je tako potrebno predstaviti kot porazdelitev akcij, npr. kot Gaussovo porazdelitev: $\mathbf{a} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma})$. V primeru determinističnega delovanja za akcijo izberemo srednjo vrednost porazdelitve, tj. $\boldsymbol{\mu}$. Ker moramo v okolju med učenjem raziskovati, je akcije takrat potrebno vzorčiti stohastično iz porazdelitve. Tu se pojavi problem, saj vzorčenje iz porazdelitve ni odvedljiva operacija in je zato v optimizaciji ne moremo uporabiti. Rešitev ponuja ponovna parametrizacija [32], ki omogoča zapis spremenljivke $\mathbf{a} = \pi_\theta(\mathbf{a}|\mathbf{s})$ kot odvedljivo

operacijo $\mathbf{a} = g_\theta(\epsilon, \mathbf{s})$. V primeru Gaussove porazdelitve lahko spremenljivko definiramo kot $\mathbf{a} = \boldsymbol{\mu} + \epsilon \cdot \boldsymbol{\sigma}$, pri čemer je ϵ naključna spremenljivka $\epsilon \sim \mathcal{N}(0,\ 1)$.

4 Metodologija

4.1 Razvoj simulatorja

Kljub vsem prednostim, ki jih spodbujevalno učenje ponuja, ima le-to tudi veliko slabost. Algoritmi spodbujevalnega učenja namreč za učenje potrebujejo veliko vzorcev oz. epizod, kar učenje v pravem okolju in v realnem času naredi precej počasno. Učenje je mogoče pohitriti z učenjem na več sistemih hkrati, vendar to zahteva dodatne stroške in otežuje postavitev celotnega sistema. Če dobre funkcije nagrade in hiperparametrov (nastavitev algoritma) še nismo odkrili, je lahko reševanje problema s spodbujevalnim učenjem praktično nemogoče.

Delo olajšajo simulirana okolja, ki lahko delujejo v pohitrenem času. Prav tako je precej enostavno postaviti več simuliranih okolij hkrati, ob primerni računalniški strojni opremi tudi nekaj 1000 okolij vzporedno.

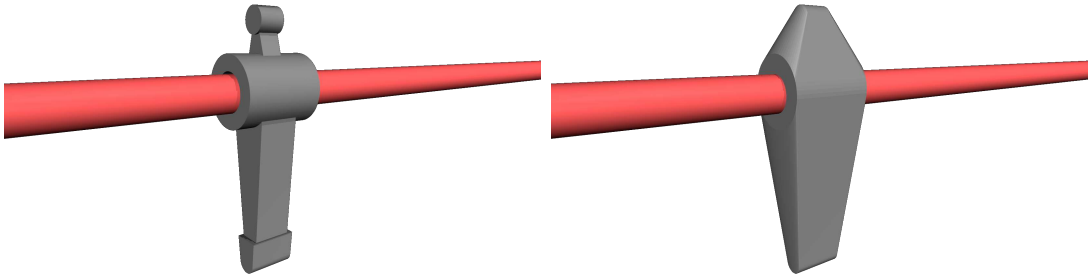
4.1.1 Izbira simulacijskega okolja

Za namen učenja strategij agentov za igranje namiznega nogometa smo pri učenju tudi sami uporabljali simulator. Na začetku smo uporabljali simulator, razvitega za namen tekmovanja FuzbAI [33]. Ta simulator deluje na podlagi simulacijskega okolja (angl. *physics engine*) PyBullet [34]. Kasneje smo, zaradi nepremostljivih programskih hroščev v okolju PyBullet, pripravili svoj simulator na podlagi simulacijskega okolja MuJoCo [35]. Oba simulatorja sta bila na začetku implementirana s programskim jezikom python [36]. Različico simulatorja, ki deluje na podlagi okolja MuJoCo, smo kasneje prepisali v programski jezik rust [37], da smo dosegli višjo hitrost delovanja. Za uporabo simulatorja v jeziku python

smo izdelali tudi pythonski vmesnik. Za izdelavo slednjega sta bila uporabljena knjižnica PyO3 [38] in orodje Maturin [39].

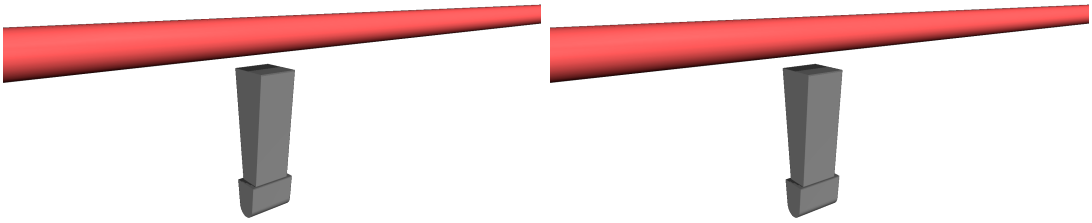
4.1.2 Pretvorba nekonveksne geometrije v konveksno

Simulacijsko okolje MuJoCo podpira računanje trkov le za konveksne geometrije [40]. To pomeni, da v simulaciji ne moremo neposredno uporabiti modelov z notranjimi koti večjimi od 180 stopinj. Če nekonveksne modele kljub temu uporabimo, MuJoCo za namen zaznavanja trkov okoli njih izdela konveksno lupino, kot prikazuje slika 4.1.



Slika 4.1: Nekonveksni 3D-model (levo) in pripadajoča konveksna lupina za zaznavanje trkov (desno)

Problem nekonveksnih 3D-modelov smo rešili tako, da smo večino prvotnih 3D-modelov iz [33], zamenjali s preprostimi modeli, ki so vgrajeni v MuJoCo. To vključuje celotno mizo, brez stranskih klančin in figur igralcev. Vsakega igralca smo iz ene nekonveksne geometrije razbili na dve konveksni. S tem smo poskrbeli, da je igralčeva konveksna lupina enaka igralčevem 3D-modelu in tako omogočili natančnejše zaznavanje in računanje trkov. Model igralca, ki smo ga razbili na dva konveksna dela, je prikazan na sliki 4.2. Na enak način smo razbili 3D-modele stranskih klančin.



Slika 4.2: Nekonveksni 3D-model igralca, razbit na dva konveksna dela (levo), in pripadajoča lupina za zaznavanje trkov (desno). Razvidno je, da lupina sovpada s 3D-modelom igralca.

4.1.3 Določitev parametrov žoge

Poleg geometrijske oblike okolja je potrebno določiti tudi fizikalne parametre žoge. Ker vseh fizikalnih parametrov žoge ne moremo neposredno meriti brez posebne opreme, smo žogi določili le znano maso in polmer. Parametre togosti smo nastavili tako, da smo dosegli najvišjo togost ob še stabilnem delovanju okolja MuJoCo. Nekatere parametre smo določili na podlagi vizualne povratne informacije. Na podlagi višine odboja po spustu smo tako v modelu trkov določili parameter dušenja, na podlagi odboja od stene in hitrosti ustavljanja pa smo določili parametre trenja.

Skladno s parametri žoge smo nastavili tudi parametre nekaterih preostalih geometrij (npr. parametre togosti igralcev).

4.1.4 Modeliranje palic

4.1.4.1 Vodenje

Prava miza palice krmili po trapeznem hitrostnem profilu. Da bi dosegli realistične odzive tudi v simulatorju, smo temu dodali motorje, vodene s P-regulatorjem s kompenzacijo Coriolisovih, centrifugalnih in težnostnih komponent. Regulator ustvarja navor (pri vrtenju palice) oz. silo (pri iztegovanju

palice) sorazmerno razliki med referenčno in dejansko hitrostjo. Referenčna hitrost se izračuna po trajektoriji konstantnega pospeška oz. pojemka z omejeno hitrostjo, posledica česar je trapezni hitrostni profil. Algoritem vodenja za eno prostostno stopnjo je prikazan v algoritmu 2, kjer spremenljivka d_{stop} označuje razdaljo, ki jo palica prepotuje pred ustavitvijo v primeru konstantnega pojemka a_{max}, T_{s} časovni korak oz. vzorčni čas simulacije, v_{max} najvišjo dovoljeno hitrost, K_{p} proporcionalno ojačanje, τ_{bias} potreben navor oz. silo za kompenzacijo Coriolisovih, centrifugalnih in težnostnih komponent ter u regulirano veličino (ciljni navor oz. silo na motorju). Za preprečevanje nihanja okoli referenčnega položaja poskrbi izraz $\sqrt{\tanh(3\,|\Delta p|\,/\,d_{\mathrm{min}})}$, ki niža proporcionalno ojačanje ob $|\Delta p| < d_{\mathrm{min}}$. Člen, ki vsebuje ojačanje K_{d}, prav tako omogoča večjo stabilnost v okolici referenčnega položaja p_{r}.

Algoritem 2 Vodenje s trapeznim profilom hitrosti

Vhod: ciljni položaj p_{r}, trenutni položaj p, trenutna hitrost v.
Izhod: ciljna hitrost v_{r}, krmilna veličina u.
Stanje: ciljna hitrost v_{r}.

Ustvarjanje trajektorije hitrosti

$\Delta p = p_{\mathrm{r}} - p$
$d_{\mathrm{stop}} = v_{\mathrm{r}}^2/(2\,a_{\mathrm{max}})$ ▷ čas ustavljanja ob začetni hitrosti v_{r} in pojemku a_{max}
if $|\Delta p| < d_{\mathrm{stop}}$ **then**
    $\Delta v_{\mathrm{r}} = -\mathrm{sgn}(v_{\mathrm{r}})\,a_{\mathrm{max}}\,T_{\mathrm{s}}$ ▷ ustavljanje s pojemkom a_{max}
else
    $\Delta v_{\mathrm{r}} = \mathrm{sgn}(\Delta p)\,a_{\mathrm{max}}\,T_{\mathrm{s}}$ ▷ pospeševanje proti p_{r} s pospeškom a_{max}
end if
$v_{\mathrm{r}} \leftarrow \mathrm{clamp}(v_{\mathrm{r}} + \Delta v_{\mathrm{r}}, -v_{\mathrm{max}}, v_{\mathrm{max}})$

Regulacija (ustvarjanje navora oz. sile)

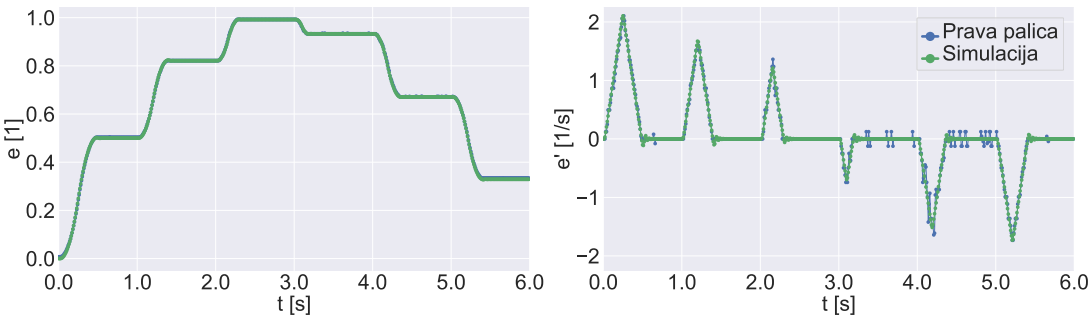
$\Delta v = v_{\mathrm{r}} - v$
$u = \sqrt{\tanh(3\,|\Delta p|\,/\,d_{\mathrm{min}})}\,K_{\mathrm{p}}\,\Delta v - K_{\mathrm{d}}\,v + \tau_{\mathrm{bias}}$

4.1.4.2 Identifikacija parametrov

Po implementaciji algoritma 2 je bilo potrebno določiti njegove parametre. Ti so omejitev hitrosti $v_{\max}$, pospešek $a_{\max}$ ter ojačanji K_{p} in K_{d}.

Parametre smo pridobili z optimizacijo, kjer smo minimizirali kriterijsko funkcijo $J = (\frac{1}{N}\sum_{n=1}^{N}(\hat{\mathbf{y}} - \mathbf{y})^2) + \alpha K_{\mathrm{p}} + \beta v_{\max}$. Vektor $\mathbf{y}$ predstavlja izmerjeni odziv sklepa prave palice, $\hat{\mathbf{y}}$ odziv v simulaciji, α in β pa regularizacijska koeficienta. Slednja se razlikujeta po vrsti sklepa. Prav tako se po vrsti sklepa razlikuje pomen $\mathbf{y}$, in sicer pri iztegu predstavlja hitrost, pri zasuku pa položaj. Razlog za uporabo položaja pri optimizaciji regulatorja zasuka izhaja iz visokih pospeškov stopenj zasuka palic. Visoki pospeški so povzročili zelo kratke pulze v hitrosti, kar je pri optimizaciji napake hitrosti vodilo do ničelnih rešitev. Regularizacija člena K_{p} se uporablja za zagotavljanje numerične stabilnosti, regularizacija člena $v_{\max}$ pa omejuje prostor rešitev na smiselne vrednosti. Začetne vrednosti parametrov $v_{\max}$ in $a_{\max}$ smo določili z odvajanjem izmerjenih odzivov položaja.

Optimizacijo kriterijske funkcije J smo izvedli z algoritmom CMA-ES (angl. *Covariance Matrix Adaptation Evolution Strategy*) [41]. Primer odziva (iztega) palice po identifikaciji parametrov prikazuje slika 4.3. Vrednosti vseh identificiranih parametrov prikazujeta tabeli 4.1 in 4.2. Parametrov regulatorja zasuka palice vratarja nismo določali z optimizacijo zaradi začetnih težav z merjenjem na pravi napravi. Po odpravi težav se je izkazalo, da sta primerni vrednosti $v_{\max}$ in $a_{\max}$ enaki tistima, ki pripadata palici obrambe.



Slika 4.3: Odziv prave (modra) in simulirane palice (zelena). Leva slika prikazuje normaliziran izteg palice (e), desna pa njegov odvod po času (e'). Prikazan je odziv obrambne palice.

Tabela 4.1: Identificirani parametri prostostne stopnje iztega

Palica	K_p [N s/m]	K_d [N s/m]	v_max [m/s]	a_max [m/s^2]
Vratar	519,864	61,023	0,8310	3,850
Obramba	511,897	17,242	1,032	3,414
Sredina	552,796	101,298	0,666	4,104
Napad	603,359	34,656	0,804	3,580

Tabela 4.2: Identificirani parametri prostostne stopnje zasuka

Palica	K_p [N m s/rad]	K_d [N m s/rad]	v_max [rad/s]	a_max [rad/s^2]
Vratar	0,2	0,05	97,040	1449,521
Obramba	0,295	0,05	97,040	1449,521
Sredina	0,369	0,05	102,938	1480,390
Napad	0,259	0,05	100,840	1482,224

4.1.5 Diskretizacija časa

Simulacijska okolja delujejo v diskretnem času oz. po korakih, s katerimi lahko le aproksimiramo zvezni čas. Eden od pomembnih parametrov, ki jih moramo določiti, je dolžina simulacijskega koraka oz. količina časa, ki mine ob integraciji. Pri izbiri moramo pretehtati med hitrostjo delovanja, natančnostjo in stabilnostjo. Ob premajhni vrednosti bo simulacija počasna, prevelike vrednosti pa bodo povzročile nepravilne odzive na trke in/ali nestabilnost simulacije.

Kljub temu, da simulacijsko okolje MuJoCo uporablja model direktne in inverzne kinematike, ki je zvezen v času, je rešitve modela (pospeške) potrebno integrirati. MuJoCo ponuja več vrst integracijskih metod, od katerih smo si v našem delu izbrali hitro različico Eulerjevega integratorja, ki je v hitrosti impliciten. Ta vrsta integratorja je bolj podrobno opisana v [42]. Velikost časovnega koraka smo v okolju MuJoCo nastavili na dve milisekundi. Za namen hitrejšega učenja v simulaciji in vodenja na pravi mizi smo v enem časovnem koraku vodenja (izvedbi ene akcije) izvedli deset korakov v okolju MuJoCo. Tako smo definirali dve vrsti časovnih korakov: nizkonivojski korak dveh milisekund (MuJoCo) in visokonivojski korak dvajsetih milisekund (izvedba akcij).

4.1.6 Prilagoditve za prenos v pravo okolje

Kasneje opisan razvoj strategij je bil v celoti izveden v simulaciji. Spodbujevalno učenje temelji na iskanju najboljših akcij v danem Markovskem odločitvenem procesu, ki je skladen s trenutnim okoljem. Če simulacija odstopa od pravega okolja, posledično tudi sam Markovski odločitveni proces odstopa od tistega, kjer se je agent učil. V okoljih, kjer je zahtevana natančna manipulacija okolja, to lahko vodi do rešitev, ki dobro delujejo le v simulaciji, v pravem okolju pa popolnoma odpovejo. Namizni nogomet je dober primer takšnega okolja, saj kompleksnejše strategije zahtevajo natančno manipulacijo žoge in tako tudi visoko natančnost same simulacije.

Najbolj primerno za prenos razvitih strategij iz simuliranega okolja v pravo okolje bi bilo začetno učenje v simulatorju in kasnejše dodatno učenje v pravem okolju. Tega smo se vzdržali iz treh večjih razlogov. Prvi je, da določenih veličin, potrebnih v nagradah, ne moremo zanesljivo meriti brez večjih sprememb sistema FuzbAI (npr. merjenje sile trkov). Drugi razlog izhaja iz narave raziskovanja spodbujevalnega učenja. Raziskovanje v algoritmu PPO je omogočeno s stohastičnim vzorčenjem Gaussove porazdelitve, kar pomeni vzorčenje akcij z Gaussovim šumom. Slednje bi povzročilo tresenje palic, kar niža življenjsko dobo mehanskih delov. Tretji večji razlog je zahteva po naključnem začetnem stanju, kjer moramo žogo postaviti na naključno mesto in ji dati naključno hitrost. Človek tega zaradi varnostnih mehanizmov sistema FuzbAI ne more početi, torej bi za to potrebovali avtomatiziran sistem postavljanja žoge v okolje oz. njenega serviranja. Za zmanjšanje količine dela smo se odločili za učenje strategij le v simulaciji, z določenimi prilagoditvami za boljši prenos v pravo okolje.

Možne težave prenosa iz simulacije v pravo okolje po učenju lahko povzročijo:

- nepravilni 3D-modeli (predvsem igralcev);
- nepravilni fizikalni parametri žoge in igralne površine (trenje, togost itd.);
- nesinhroniziranost tipal, časa vzorčenja stanj in časa izvedbe akcij;
- časovne zakasnitve med vzorčenim stanjem in pravim stanjem;
- itd.

Vse naštete težave, razen nepravilnih 3D-modelov, smo poskušali bodisi kompenzirati v pravem okolju, bodisi modelirati v simulaciji med učenjem. Na začetku je bila za boljši prenos načrtovana tudi naključna menjava 3D-modelov igralcev. Te smo se vzdržali zaradi kompleksnosti njene izvedbe. MuJoCo namreč ne podpira menjave modela simulacije brez ponovnega prevajanja le-tega in ponastavitve pripadajoče strukture stanja, kot opisano v [43]. Prav tako bi naključna menjava 3D-modelov koristila predvsem napadalcu, zato smo vpliv nepravilnosti 3D-modelov upoštevali pri razvoju strategije napadalca, pri razvoju strategij vratarja in obrambe pa nepravilnosti zanemarili.

4.1.6.1 Zmanjšanje vpliva nepravilnih fizikalnih parametrov

Da bi se izognili slabemu prenosu iz simulacije v pravo okolje zaradi nepravilno določenih fizikalnih parametrov, smo žogi dodali dušenje hitrosti, določeno z enačbo $\mathbf{f} = -\mathbf{k}_\mathrm{v} \circ \mathbf{v}$. V enačbi $\circ$ predstavlja produkt istoležnih elementov, $\mathbf{f} = [f_x, f_y]^\mathrm{T}$ silo v nasprotni smeri gibanja žoge, $\mathbf{v} = [v_x, v_y]^\mathrm{T}$ hitrost žoge in $\mathbf{k}_\mathrm{v} = [k_{\mathrm{v}x}, k_{\mathrm{v}y}]^\mathrm{T}$ pozitivni koeficient dušenja, ki se je med učenjem v posamezni epizodi naključno nastavil za osi x in y ločeno. Dušenje, ki je po oseh ločeno, dodatno simulira neenakomerno podlago igralne površine in umazanijo na žogi.

4.1.6.2 Modeliranje šuma

Meritve v simuliranem okolju so izvedene z idealnimi tipali in zato ne vsebujejo šuma. Poleg tega so tipala med seboj sinhronizirana in imajo enako vzorčno frekvenco.

Tipala v pravem svetu odstopajo od simuliranih tipal. Vračajo namreč podatke s prisotnostjo šuma, nimajo povsem točne vzorčne frekvence, ne začnejo vzorčiti ob istem času itd. Vse neidealnosti je težje ločeno simulirati, zato smo jih vse modelirali skupaj kot uniformni šum. Da bi zmanjšali občutljivost naučenih strategij na odstopanja v pravem okolju, smo uporabili amplitudo šuma, ki je višja od amplitude šuma pravih tipal.

4.2 Izdelava nasprotnika

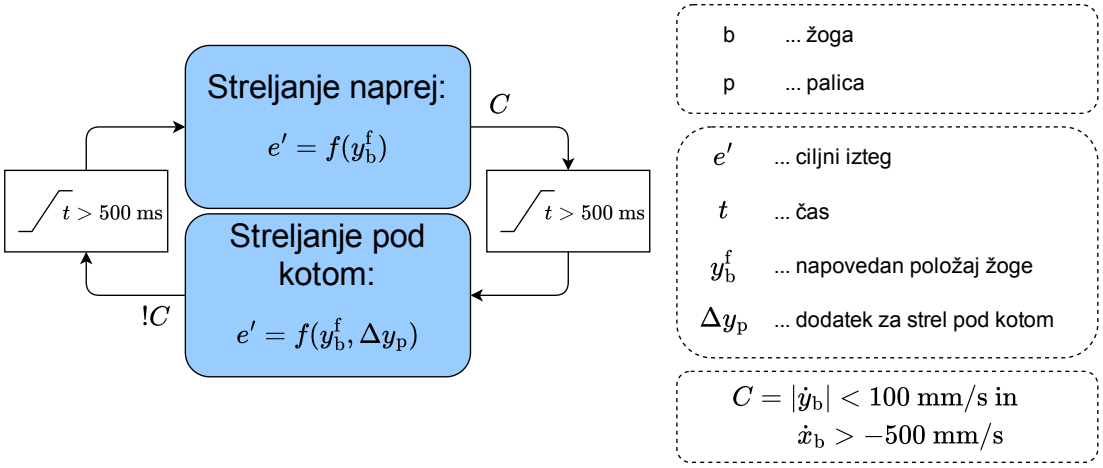
Ker je namizni nogomet igra dveh ekip, je pri učenju strategij smiselno vključiti tudi nasprotnika. V simuliranem okolju proti človeku ni mogoče igrati, zato je nasprotnika potrebno simulirati.

Ker je nasprotnik namenjen predvsem agresivnemu streljanju žoge iz različnih položajev, smo se odločili za reaktivno strategijo igranja. To pomeni, da namesto načrtovanja dolgotrajne trajektorije gibanja, agent žogo poskuša prestreči in nato čim hitreje ustreliti proti golu nasprotne ekipe.

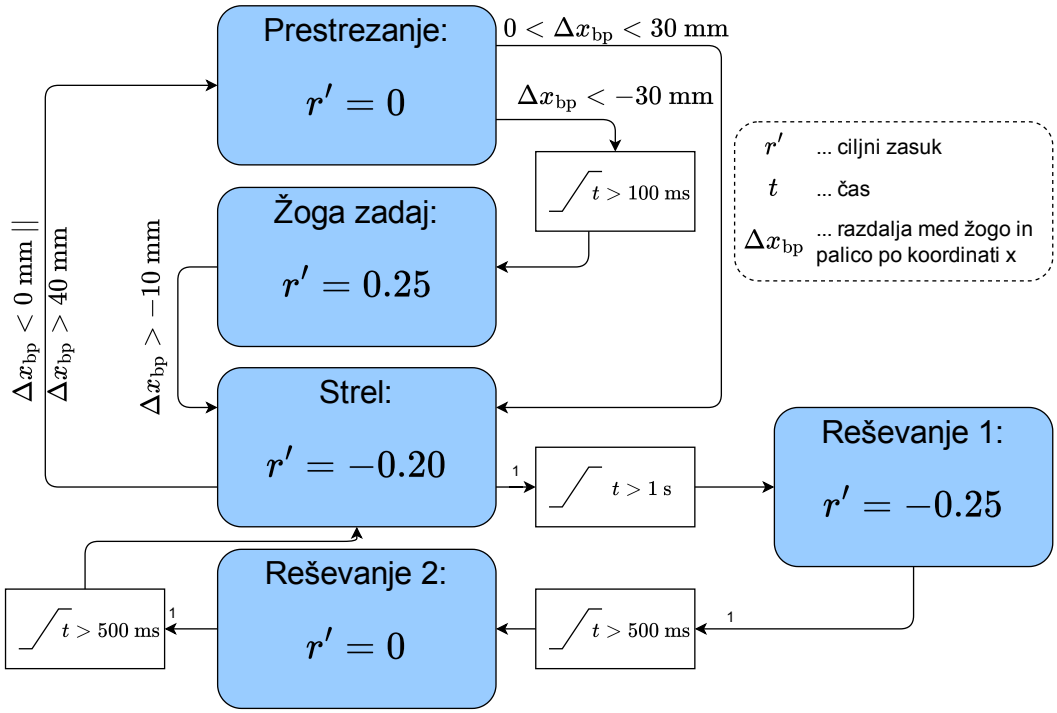
Nasprotnik je implementiran kot množica več agentov oz. palic, kjer se te razlikujejo po številu igralcev, iztegu, najvišji hitrosti itd. Sam sistem je enak za vse palice, a parametriziran za vsako palico drugače.

4.2.1 Stanja

Vsak agent nasprotnika je izveden z dvema ločenima končnima avtomatoma (angl. *finite-state machines* – FSMs). Prvi služi vodenju iztega, drugi pa vodenju zasuka. Končni avtomat za izteg je prikazan na sliki 4.4, za zasuk pa na sliki 4.5.



Slika 4.4: Končni avtomat FSM-agenta za vodenje iztega



Slika 4.5: Končni avtomat FSM-agenta za vodenje zasuka

4.2.2 Prestrezanje žoge

V obeh stanjih stopnje iztega se v osnovi izvaja prestrezanje žoge. To je izdelano kot preslikava napovedanega položaja žoge v ciljni izteg:

$$e' = e + \frac{y_{\mathrm{b}}^{\mathrm{f}} - y_{\mathrm{p}}}{E}. \tag{4.1}$$

V enačbi (4.1) e predstavlja trenutni izteg palice, e' ciljni izteg, $y_{\mathrm{b}}^{\mathrm{f}}$ koordinato y napovedanega položaja žoge, y_{p} koordinato y položaja trenutno izbranega igralca na palici, izračunanega po enačbi (4.2), in E največjo razdaljo, ki jo palica lahko prepotuje. Vse veličine v enačbi so v absolutnih enotah, razen e, ki je normirana na največji izteg palice. Tako velja $e \in [0,\ 1]$.

$$y_{\mathrm{p}} = y_{\mathrm{p}_0} + E\,e + S\,p \tag{4.2}$$

V enačbi (4.2) oznaka y_{p_0} predstavlja položaj prvega igralca na palici pri ničelnem iztegu, S razmik med igralci na palici in $p \in \{0,\ 1,\ \ldots,\ P-1\}$ indeks trenutno izbranega igralca na palici.

Če palica ne more doseči dovoljenega ciljnega iztega ($e' \notin [0{,}05,\ 0{,}95]$), se izračun tega ponovi kot opisuje algoritem 3, kjer P predstavlja število vseh igralcev na palici.

Algoritem 3 Izračun novega ciljnega iztega palice

Vhod: Koordinata y napovedanega položaja žoge y_p, relativni izteg palice e, indeks trenutno izbranega igralca p.

1: $\mathbf{\Delta y} = [y_b^\mathrm{f} - y_0,\ y_b^\mathrm{f} - y_1,\ \ldots,\ y_b^\mathrm{f} - y_{P-1}]^\mathrm{T}$
2: **for** $i = 0, 1, \ldots, P$ **do**
3: Izračun e' po enačbi (4.1).
4: **if** $e' \in [0{,}05,\ 0{,}95]$ **then**
5: **return** ▷ Položaj je ustrezen, igraj s p-tim igralcem.
6: **else**
7: $\mathbf{\Delta y}[p] \leftarrow 10\,\mathbf{\Delta y}[p]$ ▷ Prepreči ponovno izbiro p-tega igralca.
8: $p \leftarrow \operatorname{argmin}(\mathbf{\Delta y})$
9: **end if**
10: **end for**

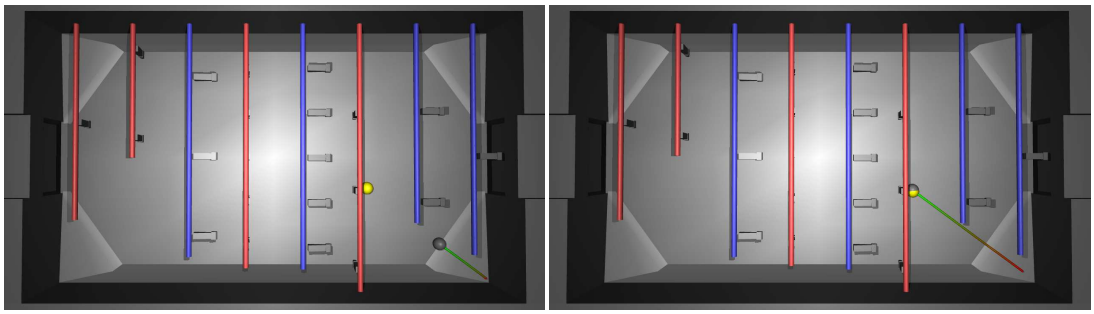
Napovedano koordinato y položaja žoge, izračunamo kot:

$$y_\mathrm{b}^\mathrm{f} = \begin{cases} y_\mathrm{b} + \dot{y}_\mathrm{b}\,\overbrace{|(\Delta x_\mathrm{bp} - X_\mathrm{b}^\mathrm{int})/\dot{x}_\mathrm{b}|}^{\text{čas do palice}}, & \text{za } |\dot{x}_\mathrm{b}| > \dot{X}_\mathrm{b}^\mathrm{int} \\ y_\mathrm{b} + \dot{y}_\mathrm{b}\,T_\mathrm{r}, & \text{drugače} \end{cases}, \tag{4.3}$$

kjer y_b predstavlja koordinato y trenutnega položaja žoge, Δx_bp razdaljo (po osi x) med žogo in palico, x_b koordinato x položaja žoge, T_r empirično določeno časovno konstanto ter $X_\mathrm{b}^\mathrm{int}$ in $\dot{X}_\mathrm{b}^\mathrm{int}$ parametra prestrezanja, katerih vrednosti so zapisane v tabeli 4.3. Primer napovedi položaja žoge prikazuje slika 4.6.

Tabela 4.3: Vrednosti parametrov za prestrezanje žoge s FSM-agentom

Parameter	Pogoj	Vrednost
$X_\mathrm{b}^\mathrm{int}$	$\Delta x_\mathrm{bp} \geq 0$	35 mm
	$\Delta x_\mathrm{bp} < 0$	−10 mm
$\dot{X}_\mathrm{b}^\mathrm{int}$	/	25 mm/s



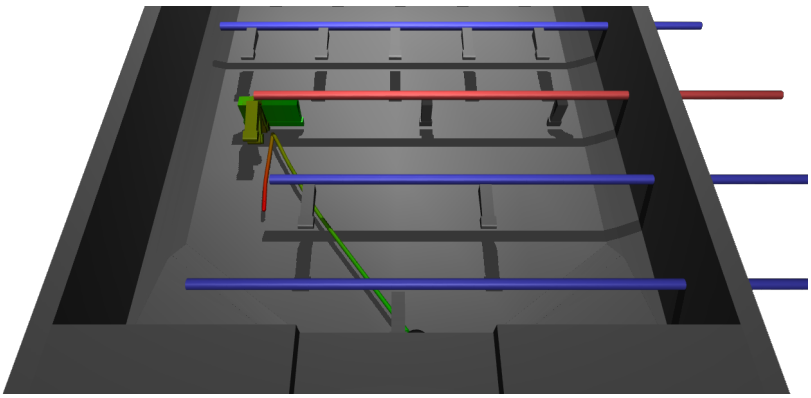
Slika 4.6: Napoved položaja žoge (rumena) in trenutni položaj (siva) pred prestrežbo (levo) ter po prestrežbi (desno)

4.2.3 Streljanje žoge

Poleg osnovnega prestrezanja žoge se v primeru stanja stranskega streljanja na gol (končni avtomat iztega) h koordinati y napovedanega položaja žoge doda zamik:

$$\Delta y_{\mathrm{p}} = \mathrm{clamp}(\frac{y_{\mathrm{b}} - W_{\mathrm{a}}/2}{L_{\mathrm{a}} - x_{\mathrm{p}}}\, r_{\mathrm{b}}\, k,\ -20\,\mathrm{mm},\ 20\,\mathrm{mm}). \tag{4.4}$$

V enačbi (4.4) y_{b} predstavlja koordinato y položaja žoge ob trku, ki jo smatramo enako koordinati y položaja igralca na palici, W_{a} širino igrišča, L_{a} dolžino igrišča, r_{b} polmer žoge, x_{p} koordinato x položaja palice in $k = 2$ popravljalno konstanto. Končni ciljni izteg izračunamo tako, da $y_{\mathrm{b}}^{\mathrm{f}}$ v enačbi (4.1) zamenjamo z $y_{\mathrm{b}}^{\mathrm{f+o}} = y_{\mathrm{b}}^{\mathrm{f}} + \Delta y_{\mathrm{p}}$. Izpeljava enačbe (4.4) je na voljo v dodatku A. Primer streljanja prikazuje slika 4.7.



Slika 4.7: Strel FSM-agenta na gol

4.3 Razvoj strategij s spodbujevalnim učenjem

4.3.1 Razvoj strategije napadalca

Tu je opisan razvoj strategije agenta za napadalno palico s spodbujevalnim učenjem. Cilj napadalne palice je streljanje žoge kot tudi prestrezanje žoge.

Končno strategijo napadalca predstavlja več zaporednih dejanj: ustavljanje žoge, ki se giblje proti napadalcu, podaja žoge med igralci in strel na gol.

Med učenjem nasprotnik ni bil prisoten, saj se je izkazalo, da njegova vključitev med učenjem vodi k slabšemu delovanju med vrednotenjem. To smo preverjali v primerjalni študiji v poglavju 5.2.1.1.

4.3.1.1 Večstopenjsko učenje

Ker se je izkazalo, da učenje celotnega zaporedja naenkrat pri višjih hitrostih ni bilo uspešno, smo učenje izvedeli s tehniko postopnega zviševanja težavnosti oz. večstopenjskega učenja (angl. *curriculum learning*) [44], ki omogoča razvoj osnovnejših sposobnosti pred izpeljanimi (npr. pred hojo moramo znati stati).

Učenje smo izvajali pri sledečih stopnjah težavnosti:

1. podajanje mirne žoge soigralcu;
2. podajanje mirne žoge soigralcu in nato pa streljanje (na gol);
3. prestrezanje počasne žoge, podajanje soigralcu in streljanje;
4. prestrezanje srednje-hitre žoge, podajanje soigralcu in streljanje ter
5. prestrezanje hitre žoge, podajanje soigralcu in streljanje.

Stopnje 2–5 z določenimi verjetnostmi vključujejo tudi pogoje prejšnjih stopenj, s čimer smo želeli ohraniti obstoječe znanje strategij pri učenju višjih stopenj in tako preprečiti prekomerno prileganje (angl. *overfitting*) na hitrejše žoge. Pogoje za prehod v višjo stopnjo težavnost podaja tabela 4.4, kjer $\overline{G}_{\text{ep}}$ predstavlja povprečni epizodni donos in k število izvedenih korakov v okolju od začetka učenja. Stopnja

je bila zvišana takoj ob izpolnjenih pogojih za zvišanje. Po zvišanju se stopnja ni več znižala.

Tabela 4.4: Pogoji za zvišanje stopnje težavnosti pri učenju napadalca

Prehod težavnosti	Pogoj
$1 \rightarrow 2$	$\overline{G}_{\mathrm{ep}} \geq 0{,}5$
$2 \rightarrow 3$	$k \geq 60 \times 10^6$
$3 \rightarrow 4$	$k \geq 100 \times 10^6$
$4 \rightarrow 5$	$k \geq 200 \times 10^6$

4.3.1.2 Epizoda

Epizoda je bila definirana različno za različne težavnostne stopnje, z nekaj skupnimi končnimi in okrnjenimi stanji. Pri vseh stopnjah se je epizoda štela za okrnjeno, če je žoga obstala na mestu ali pa je preteklo 250 korakov (5 sekund). Pri vseh stopnjah se je epizoda štela za končano, če je za žogo veljalo $\dot{x}_{\mathrm{b}} < 0\,\mathrm{m/s}$ in $x_{\mathrm{b}} < 750\,\mathrm{mm}$ ali pa je žoga zapustila veljavno igralno območje. Epizoda se je dodatno končala, če je:

- v primeru težavnostne stopnje 1: koordinata x položaja žoge presegla 910 mm ali pa je agent uspešno podal žogo od enega igralca do drugega;
- v primeru težavnostne stopnje 2–5: žoga pristala v nasprotnikovem golu.

Smiselno je omeniti tudi, kako se je epizoda začela. Palica napadalca je bila ob začetku epizode vedno naključno postavljena po uniformni porazdelitvi, in sicer po iztegu $e \sim \mathcal{U}(0\,\%,\ 100\,\%)$ in zasuku (v radianih) $r \sim \mathcal{U}(0{,}5\pi,\ 0{,}5\pi)$. Začetna postavitev žoge in njena hitrost sta bili odvisni od trenutne težavnostne stopnje. Posamezna težavnostna stopnja je imela več možnih območij konfiguracij, tj. območij položajev in hitrosti. Vsako območje konfiguracij je bilo izbrano z neko verjetnostjo, dejanska konfiguracija pa je bila vzorčena iz uniformne porazdelitve v izbranem območju. Območja začetnih konfiguracij žoge definira tabela 4.5, pri čemer koordinati x_{b} in y_{b} predstavljata položaj žoge. Verjetnosti za izbiro posameznega območja pri določeni stopnji podaja tabela 4.6.

Tabela 4.5: Območja začetnih konfiguracij žoge pri učenju napadalca

Območje	x_b[mm]	y_b[mm]	$\dot{x}_\mathrm{b}$[m/s]	$\dot{y}_\mathrm{b}$[m/s]
1	[820, 860]	[100, 600]	[−0,01, 0,01]	[−0,10, 0,10]
2	[880, 940]	[100, 600]	−[0,20, 0,08]	[−0,08, 0,08]
3	[720, 780]	[100, 600]	[0,08, 0,20]	[−0,08, 0,08]
4	[950, 1100]	[100, 600]	−[0,35, 0,20]	[−0,20, 0,20]
5	[560, 710]	[100, 600]	[0,20, 0,35]	[−0,20, 0,20]
6	[1000, 1100]	[100, 600]	−[0,50, 0,35]	[−0,50, 0,50]
7	[560, 600]	[100, 600]	[0,35, 0,50]	[−0,50, 0,50]

Tabela 4.6: Verjetnosti izbire območja konfiguracije za posamezno težavnostno stopnjo

Težavnost	Verjetnost	Območje
1 & 2	1,00	1
3	0,30	1
	0,35	2
	0,35	3
4	0,20	1
	0,10	2
	0,10	3
	0,30	4
	0,30	5
5	0,20	1
	0,10	2
	0,10	3
	0,10	4
	0,10	5
	0,20	6
	0,20	7

4.3.1.3 Prostor stanj

Stanje smo napadalcu definirali kot vektor, ki vsebuje:

- položaj žoge po koordinatah x in y,
- izteg palice $e \in [0,\ 1]$ ter
- zasuk palice $r \in [-64,\ 64]$.

Območje zasuka palice r ustreza polnem zasuku v vsako smer, tj. $[-2\pi,\ 2\pi]$ radianov.

Vektor je bil dodatno standardiziran, tj. transformiran z:

$$\mathbf{z} = (\mathbf{s} - \boldsymbol{\mu}) \oslash \boldsymbol{\sigma}. \tag{4.5}$$

V enačbi (4.5) $\mathbf{z}$ predstavlja standardizirano vrednost stanja, $\mathbf{s}$ vektor stanja, $\boldsymbol{\mu}$ srednjo vrednost stanja, $\boldsymbol{\sigma}$ standardni odklon stanja in $\oslash$ deljenje istoležnih elementov. Vrednosti $\boldsymbol{\mu}$ in $\boldsymbol{\sigma}$ smo med učenjem računali sproti s tekočim povprečjem in Welfordovim algoritmom [45] na podlagi vseh preteklih (nestandardiziranih) stanj, pri čemer smo uporabljali implementacijo standardizacije iz knjižnice Ray RLlib. Vrednost $\mathbf{z}$ je bila uporabljena kot vhod v nevronsko mrežo strategije. Nevronsko mrežo je sestavljal navadni MLP, ki mu je sledila plast LSTM. Slednja je poskrbela za pomnjenje časovnih informacij, ki so bile izluščene iz preteklih stanj. Aktivacijska funkcija v skritih plasteh in izhodni plasti MLP je bila hiperbolični tangens.

4.3.1.4 Prostor akcij

Akcije smo definirali kot relativni premik glede na trenutni položaj. Akcije so bile za napadalca definirane kot $a^{(\mathrm{a})} = [a_{\mathrm{e}}^{(\mathrm{a})},\ a_{\mathrm{r}}^{(\mathrm{a})}]$, pri čemer $a_{\mathrm{e}}^{(\mathrm{a})}$ predstavlja relativni premik iztega in $a_{\mathrm{r}}^{(\mathrm{a})}$ relativni premik zasuka. Oba elementa sta bila omejena na območje $[-0{,}5,\ 0{,}5]$. Negativne akcije zasuka v primeru začetne postavitve palice navzdol (0 rad) pomenijo brc proti nasprotnikovemu golu, pozitivne pa proti lastnemu golu. Dejanski ukaz, ki smo ga poslali sistemu FuzbAI (in simulatorju), je bil definiran kot $e_{\mathrm{target}}^{(\mathrm{a})} = \mathrm{clamp}(e^{(\mathrm{a})} + a_{\mathrm{e}}^{(\mathrm{a})}, 0, 1)$ za izteg in $r_{\mathrm{target}}^{(\mathrm{a})} = \mathrm{clamp}(r^{(\mathrm{a})}/64 + a_{\mathrm{r}}^{(\mathrm{a})},\ -0{,}2,\ 0{,}5)$ za zasuk. Območje ukaza ciljnega iztega

$[0,\ 1]$ ustreza iztegu $[0,\ 100]$ %, območje ukaza ciljnega zasuka $[-1,\ 1]$ pa zasuku $[-2\pi,\ 2\pi]$ radianov.

4.3.1.5 Nagrada

Neodvisno od stopnje težavnosti je osnovna funkcija nagrade definirana kot:

$$R_{\text{base}}^{(\text{a})} = \underbrace{-0{,}010\,(r/64)^2}_{\text{drži palico proti tlem}} - \underbrace{0{,}05\,\left(a_{\text{r}}^{(\text{a})}\right)^2}_{\substack{\text{stopnjo zasuka}\\ \text{čim manj premikaj}}}\,, \tag{4.6}$$

kjer r predstavlja zasuk palice in a_r akcijo zasuka. Prvi člen, ki spodbuja držanje palice proti tlem, omogoča boljše raziskovanje pri podajanju in sprejemanju žoge. Drugi člen spodbuja porabo čim manj energije pri zasukih in tako preprečuje oscilacije.

Osnovna funkcija nagrade je dodatno razširjena glede na stopnjo težavnosti. Pri stopnji težavnosti 1 je skupna funkcija nagrade definirana kot:

$$R_{\text{pass}}^{(\text{a})} = R_{\text{base}}^{(\text{a})} + P_{\text{first}} \underbrace{\left(1 - \min\left(\left(\frac{x_{\text{c}}^{\text{b}} - 830\,\text{mm}}{100\,\text{mm}}\right)^2, 1\right)\right)}_{\text{podaj čim bližje palici}}, \tag{4.7}$$

kjer x_{c}^{b} predstavlja koordinato x žoge ob dotiku s soigralcem in $P_{\text{first}} \in \{0,\ 1\}$ indikator, ki zasede vrednost 1 ob prvi uspešni podaji in vrednost 0 v preostalih primerih. Člen, označen s „podaj čim bližje palici”, agenta spodbudi k podajam, ki so čim bližje položaju palice po koordinati x. Z njim smo poskušali izboljšati prenos v pravo okolje, kjer oblike igralcev nekoliko odstopajo od igralcev v simulaciji.

Pri stopnjah težavnosti 2–5 je skupna funkcija nagrade definirana kot:

$$R_{\text{shot}}^{(\text{a})} = R_{\text{pass}}^{(\text{a})} + M\,(2\,S + \underbrace{0{,}010\,(f_x + \min(\max(f_z, -f_x), 0))}_{\text{čim močneje udari žogo, brez tiščanja v tla}}) \tag{4.8}$$

kjer $S \in \{0,\ 1\}$ predstavlja indikator o zadetem golu nasprotnika, $M \in \{0,\ 1\}$ indikator o tem ali je bil zadnji dotik na žogo izveden s sredinskim igralcem napadalčeve palice ter f_x in f_z silo na žogo (kot posledico sredinskega igralca)

v smeri osi x oz. z. Dodatek f_x k nagradi poskrbi, da so trki v smeri osi x čim močnejši. Z njim smo želeli zvišati možnost rešitev, kjer bi agent žogo proti nasprotnikovem golu ustrelil z največjo možno silo. Dodatek f_z, ki je ob pritisku žoge v tla negativen, prepreči izkoriščanje pozitivne nagrade, ki se zgodi zaradi člena f_x ob tiščanju žoge v tla.

4.3.1.6 Hiperparametri

Hiperparametri uravnavajo delovanje algoritmov. Ker je kvaliteta naučenih strategij precej pogojena s hiperparametri algoritma spodbujevalnega učenja, je poleg določitve funkcije nagrade, prostora akcij in prostora stanj smiselno določiti tudi dobre hiperparametre.

Najboljše hiperparametre bi našli z izčrpnim iskanjem, kjer bi večkrat preizkusili vse možne vrednosti hiperparametrov. Izčrpno iskanje je zaradi časovnih zahtev v praksi uporabno le pri manjšem številu kombinacij vrednosti hiperparametrov, v našem primeru pa praktično nemogoče, saj algoritem PPO vsebuje nemalo število hiperparametrov. Dodatno so ti večinoma zvezni, kar v primeru izčrpnega iskanja zahteva predhodno diskretizacijo.

Hiperparametre algoritma PPO smo pri učenju napadalca najprej poskusili določiti z vzorčenjem velikega števila vzorcev. V našem primeru smo vzeli 1000 vzorcev hiperparametrov in upoštevali tisti vzorec, ki je dal največji končni epizodni donos. Da bi zmanjšali skupen čas iskanja, smo strategijo učili le pri stopnji težavnosti 1 (podajanje žoge iz mirnega začetka).

Izkazalo se je, da so pridobljeni hiperparametri, kljub dobrim rezultatom pri podajanju žoge, izkazovali precej slabše rezultate pri učenju višjih stopenj težavnosti. Ker tudi ponovni poskusi optimizacije hiperparametrov niso izkazovali boljših rezultatov, smo učenje izvedli še s populacijskim učenjem (angl. *Population-Based Training* – PBT) [46].

Algoritem PBT hiperparametre optimizira sproti med samim učenjem, pri čemer posamezni poskusi delujejo vzporedno drug na drugega. Periodično se najslabši del množice poskusov zamenja s kopijami najboljšega dela. Kopije se

nato perturbira (npr. pomnoži z neko konstanto) ali pa na novo vzorči. V našem primeru je kritična tudi lastnost, da algoritem PBT lahko najde različne hiperparametre ob različnih stopnjah težavnostih.

Konstantne hiperparametre, ki smo jih nastavili ročno ali pa vzeli iz preteklih poskusov optimizacije hiperparametrov, podaja tabela 4.7. V tabeli N_{seq} predstavlja število časovnih korakov vključenih v sekvenco LSTM med učenjem; N_{layer} število plasti MLP; N_{unit} število vrstic matrike uteži v posamezni plasti (število nevronov); N_{batch} število časovnih korakov vzorčenih v okolju za posamezno iteracijo algoritma PPO; B_{SGD} število vzorcev uporabljenih za oceno ciljne funkcije algoritma PPO in γ faktor popusta (angl. *discount factor*) nagrade.

Tabela 4.7: Vrednosti konstantnih hiperparametrov pri učenju napadalca

Parameter	Vrednost
N_{seq}	30
N_{layer}	4
N_{unit}	256
N_{batch}	32768
B_{SGD}	4096
γ	0,99
...	privzeta vrednost v Ray RLlib

Hiperparametre, ki smo jih optimizirali z algoritmom PBT, podaja tabela 4.8. V tabeli N_{iter} predstavlja število iteracij optimizacije z istimi vzorci iz okolja, α hitrost učenja (angl. *learning rate*), ε faktor rezanja gradientov (angl. *clipping factor*), λ parameter algoritma GAE in h faktor entropijskega bonusa. Pri optimizaciji hiperparametrov z algoritmom PBT smo uporabili perturbacijska faktorja 0,8 in 1,2.

4.3.2 Razvoj strategije vratarja in strategije obrambe

Vratar in obramba imata v osnovi enako vlogo, tj. branjenje gola. Učenje strategij je potekalo v dveh korakih. Najprej smo učili strategijo vratarja, v prisotnosti nasprotnika in brez prisotnosti drugih palic v lastni ekipi. Po končanem uče-

Tabela 4.8: Prostor za vzorčenje hiperparametrov pri učenju napadalca, ki smo jih optimizirali z algoritmom PBT. Prostor predstavlja meje uniformne porazdelitve ob začetku učenja (inicializacija) in ob ponovnem vzorčenju v algoritmu PBT (med učenjem).

Parameter	**Območje vzorčenja**	
	Inicializacija	Med učenjem
N_{iter}	$\{8, 9, \ldots, 12\}$	$\{3, 4, \ldots, 15\}$
α	[0,000 01, 0,000 05]	[0,000 01, 0,001]
ε	[0,3, 0,4]	[0,005, 0,5]
λ	[0,9, 1,0]	[0,5, 1,0]
h	[0,0, 0,005]	[0,0, 0,002]

nju strategije vratarja smo začeli učiti strategijo obrambe. Pri tem je bil vratar postavljen v okolje, kjer je deloval deterministično in se ni več učil. Strategija obrambe se je tako učila odpravljanja slabosti strategije vratarja, s čimer smo poskušali doseči sodelovanje med agentoma. Prav tako je bil med učenjem prisoten nasprotnik.

Pri učenju obeh strategij se je nasprotnik delno spreminjal. Nasprotnikove palice vratarja, obrambe in srednje vrste so bile krmiljenje s FSM-agentom, palica napadalca pa je menjala med FSM-agentom in RL-agentom. Učenje slednjega je opisano v poglavju 4.3.1.

4.3.2.1 Epizoda

Epizoda se je pri učenju začela z naključno postavitvijo žoge v okolje. Poleg začetnega položaja smo žogi določili tudi začetno hitrost. To smo dobili tako, da smo najprej iz uniformne porazdelitve vzorčili hitrost po osi x in ciljni položaj v golu, kjer žoga brez posredovanju katerekoli palice konča. Potem smo z enačbo (4.9) izračunali pripadajočo hitrost po osi y.

$$\dot{y}^b = \frac{y^{\mathrm{b}}_{\mathrm{target}} - y^{\mathrm{b}}}{t_{\mathrm{x}^{\mathrm{b}} \to \mathrm{x}^{\mathrm{b}}_{\mathrm{target}}}} = \frac{y^{\mathrm{b}}_{\mathrm{target}} - y^{\mathrm{b}}}{x^{\mathrm{b}}_{\mathrm{target}} - x^{\mathrm{b}}}\, \dot{x}^{\mathrm{b}} \tag{4.9}$$

V enačbi (4.9) x^{b} in y^{b} predstavljata vzorčene začetne položaje žoge, $\dot{x}^{\mathrm{b}}$ vzorčeno začetno hitrosti žoge po osi x, $y^{\mathrm{b}}_{\mathrm{target}}$ vzorčen ciljni položaj po osi y, $x^{\mathrm{b}}_{\mathrm{target}}$ ciljni položaj po osi x, ki je v našem primeru 0, ter t čas. Območja veličin podaja tabela 4.9.

Tabela 4.9: Območja vzorčenja vrednosti parametrov začetne konfiguracije žoge pri učenju strategije vratarja oz. strategije obrambe. Vrednosti so vzorčene iz uniformne porazdelitve.

Veličina	x^{b} [mm]	y^{b} [mm]	$y^{\mathrm{b}}_{\mathrm{target}}$ [mm]	$\dot{x}^{\mathrm{b}}$ [m/s]
Območje	[450, 750]	[100, 600]	[250, 450]	−[0,25, 3,0]

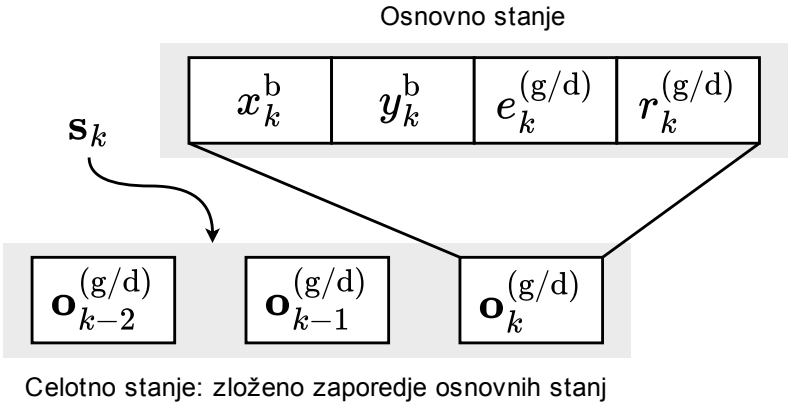
Epizoda se šteje za okrnjeno, če preteče 250 korakov (5 sekund) ali pa žoga obstoji na mestu. Za končano se epizoda šteje ob neuspešnem branjenju žoge ali ob žogi izven igralnega polja.

4.3.2.2 Prostor stanj

Strategija vratarja kot tudi strategija obrambe imata obe lastno stanje, ki je po zasnovi enako. Osnovno stanje je definirano kot vektor, ki vsebuje:

- položaj žoge po koordinatah x in y,
- izteg palice $e \in [0,\ 1]$ ter
- zasuk palice $r \in [-64,\ 64]$,

torej je enako zasnovi stanja napadalčeve strategije. Da bi v stanje vključili časovne informacije, smo celotno stanje dobili z zlaganjem zaporednih preteklih osnovnih stanj v en razširjen vektor. Eksperimentalno smo določili, da najboljše rešitve dobimo z zlaganjem zadnjih treh osnovnih stanj. Stanje vratarja oz. obrambe je ponazorjeno s sliko 4.8, kjer k označuje trenutni korak (dolg dvajset milisekund), x^{b} in y^{b} položaj žoge, e izteg palice in r zasuk palice. Celotno stanje je bilo pred vhodom v nevronsko mrežo standardizirano po enačbi (4.5). Standardizirano stanje je bilo uporabljeno kot vhod v nevronsko mrežo strategije. Nevronska mreža je bila navaden MLP. Aktivacijska funkcija slednjega je bila v skritih plasteh in izhodni plasti hiperbolični tangens.



Slika 4.8: Stanje za strategijo vratarja oz. obrambe

4.3.2.3 Prostor akcij

Prostor akcij je za strategijo vratarja in tudi strategijo obrambe definiran enako kot za strategijo napadalca (glej poglavje 4.3.1.4). Drugačen je le ukaz, ki se ga pošlje sistemu FuzbAI. Ta se razlikuje v mejnih vrednostih stopnje zasuka. Ukaz iztega je za agenta vratarja in obrambe definiran kot $e_{\mathrm{target}}^{(g/d)} = \mathrm{clamp}(e^{(g/d)} + a_{\mathrm{e}}^{(g/d)},\ 0,\ 1)$. Ukaz zasuka je za agenta vratarja definiran kot $r_{\mathrm{target}}^{(g)} = \mathrm{clamp}(r^{(g)}/64 + a_{\mathrm{r}}^{(g)},\ -0{,}1,\ 0{,}15)$, za agenta obrambe pa kot $r_{\mathrm{target}}^{(d)} = \mathrm{clamp}(r^{(d)}/64 + a_{\mathrm{r}}^{(d)},\ -0{,}1,\ 0{,}5)$.

4.3.2.4 Nagrada

Kljub skupnim ciljem se funkcija nagrade rahlo razlikuje med strategijo vratarja in strategijo obrambe. Struktura je enaka, razlikujejo pa se uteži pred posameznimi členi.

Funkcija nagrade je definirana kot:

$$\begin{aligned} R^{(\mathrm{g/d})} = &- \overbrace{w_e\,(e^{(\mathrm{g/d})} - 0{,}5)^2}^{\text{drži se sredine}} - \overbrace{w_r\,(r^{(\mathrm{g/d})}/64)^2}^{\text{drži palico navzdol}} - \overbrace{w_{a_r}\,(a_{\mathrm{r}}^{(\mathrm{g/d})})^2}^{\substack{\text{stopnjo zasuka}\\ \text{čim manj premikaj}}} \\ &+ \underbrace{w_f\,(f_x^{(\mathrm{g/d})} + \min(\max(f_z^{(\mathrm{g/d})}, -f_x^{(\mathrm{g/d})}), 0))}_{\text{čim močneje udari žogo, brez tiščanja v tla}} - w_G\,G, \end{aligned} \tag{4.10}$$

kjer G predstavlja indikator o prejetem golu, e izteg palice, r zasuk palice, a_r akcijo zasuka ter f_x in f_z silo na žogo po oseh x in z ob trku med žogo in igralci pripadajoče palice. Vrednosti uteži podaja tabela 4.10.

Tabela 4.10: Vrednosti uteži v funkciji nagrade za strategijo vratarja oz. obrambe

Utež	Vratar	Obramba
w_e	0,0025	0,005
w_r	0,050	0,010
w_{a_r}	0,025	0,010
w_f	0,010	0,030
w_G	2,000	2,000

4.3.2.5 Hiperparametri

Optimizacijo hiperparametrov smo za strategijo vratarja in strategijo obrambe izvedli z algoritmom TPE (angl. *Tree-structured Parzen Estimator*) [47], kjer smo uporabili implementacijo iz knjižnice Optuna [48]. Poskusi so med optimizacijo lahko dosegli največ 15 milijonov korakov v okolju (šteto od začetka učenja). Uporabljene hiperparametre za učenje strategije vratarja in strategije obrambe podaja tabela 4.11, kjer N_{layer} predstavlja število skritih plasti MLP, N_{unit} število vrstic matrike uteži v posamezni plasti, α hitrost učenja, ϵ faktor rezanja gradientov, N_{iter} število iteracij optimizacije z istimi vzorci iz okolja, λ parameter algoritma GAE, h faktor entropijskega bonusa, N_{batch} število časovnih korakov vzorčenih v okolju pred novim učenjem, B_{SGD} število vzorcev uporabljenih za oceno ciljne funkcije algoritma PPO in γ faktor popusta nagrade.

4.3.3 Zmanjšanje vpliva časovne zakasnitve stanja na delovanje strategij

Opis razvoja strategij je temeljil na okolju, kjer stanje ni časovno zakasnjeno. Tu so opisani koraki, ki smo jih izvedli za namen prenosa razvitih strategij v zakasnjeno okolje.

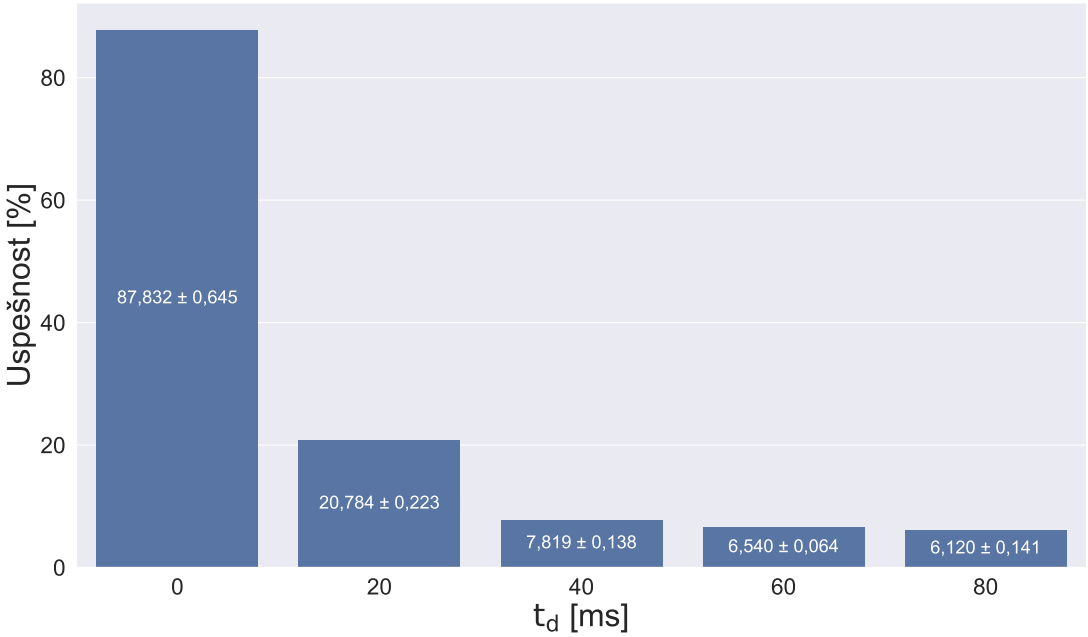
Tabela 4.11: Hiperparametri uporabljeni za učenje strategij vratarja in obrambe

Hiperparameter	Vratar	Obramba
N_{layer}	2	2
N_{unit}	256	256
α	0,0002	0,000 546
ϵ	0,100	0,1337
N_{iter}	11	10
λ	0,80	0,79
h	0,0002	0,0002
N_{batch}	32768	32768
B_{SGD}	1024	2048
γ	0,997 65	0,975 62
...		privzeta vrednost v RLlib

V pravem okolju so meritve zakasnjene. Zakasnitve so posledica komunikacije med komponentami, časa procesiranja slik kamer, neusklajenosti časa akcij in časa zajema ipd. Ker stanje strategij sestavimo iz meritev, je posledično tudi samo stanje, ki ga vidi strategija, zakasnjeno. V okviru Markovskega odločitvenega procesa časovne zakasnitve stanja v splošnem ni mogoče zanemariti, saj ta krši Markovsko lastnost. Naslednje stanje namreč v zakasnjenem okolju ni odvisno le od trenutnega stanja in akcije, temveč tudi od vseh akcij od časa $t - t_{\text{d}}$ naprej, kjer je t_{d} časovna zakasnitev. Posledično je učenje s klasičnimi pristopi v takih okoljih močno oteženo.

V primeru majhnih zakasnitev bi strategijo lahko učili v nezakasnjenem okolju in jo po učenju brez večjih posledic prenesli v zakasnjeno okolje. Neposrednega prenosa v našem primeru nismo mogli storiti, saj zakasnitev stanja v pravem okolju znaša približno 40 milisekund. Zakasnitev se morda zdi nizka, vendar ob upoštevanju možnih hitrosti žoge ter velikosti žoge in igralcev časovna zakasnitev ni zanemarljiva. Na primer, ob zakasnitvi $t_{\text{d}} = 40\,\text{ms}$ in hitrosti žoge $v = 1\,\text{m/s}$ napaka položaja ($\Delta p = v \cdot t_{\text{d}}$) znaša 40 mm. Če upoštevamo širino igralcev, ki je 20 mm, napaka po širini igrišča tako ustreza dvema igralcema. Odprava vpliva časovne zakasnitve na stanju je torej nujna. To potrjujejo poskusi podaj in zatem strelov na gol, s strani napadalca učenega v nezakasnjenem okolju, kot kaže

slika 4.9. Slika prikazuje uspešnost podaj in zatem strelov na gol, za različne vrednosti časovnega zamika. Vrednosti predstavljajo povprečje ± negotovost petih poskusov, kjer je bila žoga naključno postavljena med igralce palic. Vsak poskus je trajal 5000 epizod.



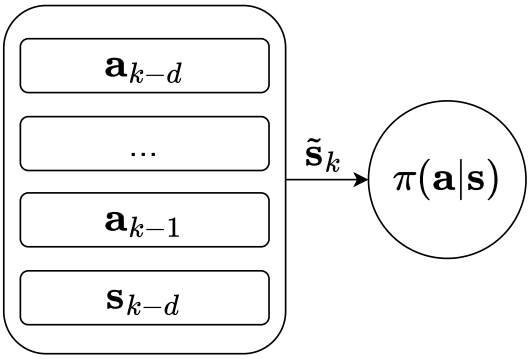
Slika 4.9: Vpliv časovne zakasnitve stanja (t_d) na uspešnost podaj in zatem strelov na gol. Prikazan je rezultat za strategijo napadalca iz časa začetnega razvoja.

Dodatni zamiki se pojavijo pri izračunu akcij zaradi časa inference nevronskih mrež, mrtvega časa palic in komunikacije. Te v našem delu obravnavamo kot del zakasnitev stanja.

4.3.3.1 Učenje v zakasnjenem okolju

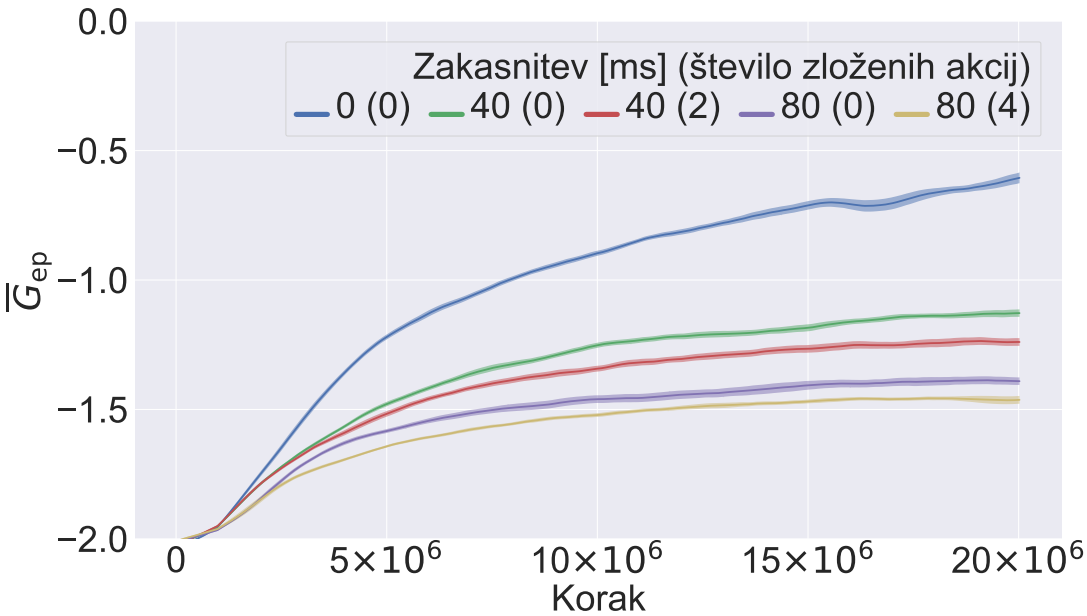
Klasičen pristop za obnovitev Markovske lastnosti je zlaganje d zadnjih akcij skupaj k trenutnemu stanju [49], pri čemer je d časovna zakasnitev v enotah izvedenih korakov v okolju. Časovno zakasnitev t_d je mogoče izračunati iz zakasnitve d z enačbo $t_\mathrm{d} = T_\mathrm{s}\, d$, kjer je $T_\mathrm{s} = 20\,\mathrm{ms}$ dolžina enega koraka oz. čas vzorčenja.

Akcije so zlagane in dodane k stanju kot prikazuje slika 4.10. Kljub temu,



Slika 4.10: Zlaganje akcij z namenom izpolnitve Markovske lastnosti. Novo stanje ($\widetilde{\mathbf{s}}_k$) je vektor, ki združuje d zadnjih akcij in trenutno meritev, ki predstavlja zakasnjeno stanje ($\mathbf{s}_{k-d}$).

da pristop zlaganja akcij deluje, je precej daleč od optimalnega, kar prikazuje slika 4.11. Pravzaprav se v našem primeru izkaže, da je učenje v zakasnjenem okolju bolj uspešno ob nezlaganju akcij, vendar še vedno precej slabše kot učenje v nezakasnjenem okolju.



Slika 4.11: Povprečni epizodni donos ($\overline{G}_{\text{ep}}$) med učenjem vratarja, katerega stanje vključuje več akcij in tri osnovna stanja ter deluje v okolju z zakasnjenim osnovnim stanjem. Vrednost $\overline{G}_{\text{ep}} = -2$ približno ustreza ničelni uspešnosti branjenja, vrednost $\overline{G}_{\text{ep}} = 0$ pa približno ustreza popolni uspešnosti branjenja.

Da bi se izognili neoptimalnim strategijam, smo učenje teh najprej izvedli v nezakasnjenem okolju, kot je opisano v prejšnjih poglavjih. Po konvergenci povprečnega epizodnega donosa smo nato posegli po modelih za kompenzacijo zakasnitev in učenje nadaljevali v zakasnjenem okolju.

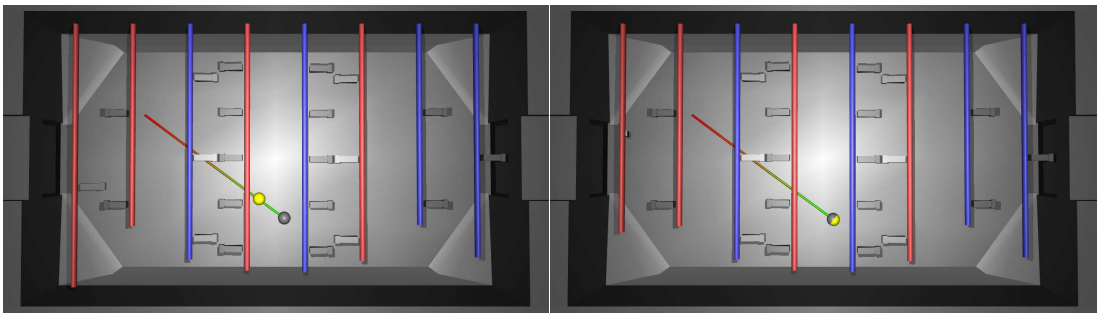
4.3.3.2 Odprava vpliva zakasnitve na položaju žoge

Pri ocenjevanju položaja žoge na podlagi zakasnjenega stanja smo izkoristili dejstvo, da se hitrost žoge (brez zunanjih vplivov) spreminja precej počasi. Tako je hitrost žoge na kratkem časovnem intervalu lahko obravnavana kot konstanta.

Oceno nezakasnjenega položaja žoge izračunamo kot:

$$\hat{\mathbf{p}}_t = \mathbf{p}_{t-t_\mathrm{d}} + \mathbf{v}_{t-t_\mathrm{d}} \cdot t_\mathrm{d}, \tag{4.11}$$

kjer $\mathbf{p}_{t-t_\mathrm{d}}$ predstavlja zakasnjen položaj, $\mathbf{v}_{t-t_\mathrm{d}}$ zakasnjeno hitrost in t_d časovno zakasnitev. Model iz enačbe (4.11) ni popoln, saj enačba ne upošteva trkov med žogo in igralci. V primeru slednjega bo ocena napačna t_d časa, vse do videne posledice na hitrosti žoge. Kljub temu smo to vrsto kompenzacije obdržali, saj je računsko učinkovita kot tudi enostavna za implementacijo in uravnavanje. Primer kompenzacije zakasnitve na položaju žoge prikazuje slika 4.12.



Slika 4.12: Ocenjen položaj žoge (označen z rumeno barvo) pred kompenzacijo (levo) in po kompenzaciji (desno). Pravi položaj žoge je označen s sivo barvo.

4.3.3.3 Ocenjevanje hitrosti

Iz enačbe (4.11) je razvidno, da pri izračunu ocenjenega položaja žoge potrebujemo tudi informacijo o hitrosti žoge. To sistem FuzbAI že ocenjuje, vendar se njena vrednost posodobi vsakih 80 milisekund, kar ni idealno ob trkih žoge (npr. strel nasprotnika). V ta namen smo razvili alternativen način ocenjevanja hitrosti, ki temelji na odvajanju in dvakratnemu glajenju surovih odvodov.

Izračun hitrosti poteka v treh korakih. Prvi korak je odvajanje koordinat x in y po času z metodo končnih razlik ($\dot{p} = (p_t - p_{t-1}) \ / \ T_\mathrm{s}$). Drugi korak je začetno glajenje s filtrom prvega reda, kjer iz signala hitrosti odstranimo višje konice šuma. Tretji korak je končno glajenje z drugim filtrom prvega reda, kjer bolj kvaliteten, vendar še vedno pošumljen signal, dodatno zgladimo. Da bi preprečili vnos časovnih zakasnitev v izračun hitrosti, smo obema filtroma določili zgornjo mejo dovoljene spremembe hitrosti, nad katero se stanje filtra ponastavi na trenutno surovo vrednost odvoda. Glajenje z zgornjo mejo je opisano z:

$$\overline{v} \leftarrow \begin{cases} \overline{v} + (\dot{p} - \overline{v})\, c, & \text{za } |\dot{p} - \overline{v}| < 2\,\sigma \\ \dot{p}, & \text{drugače} \end{cases}, \tag{4.12}$$

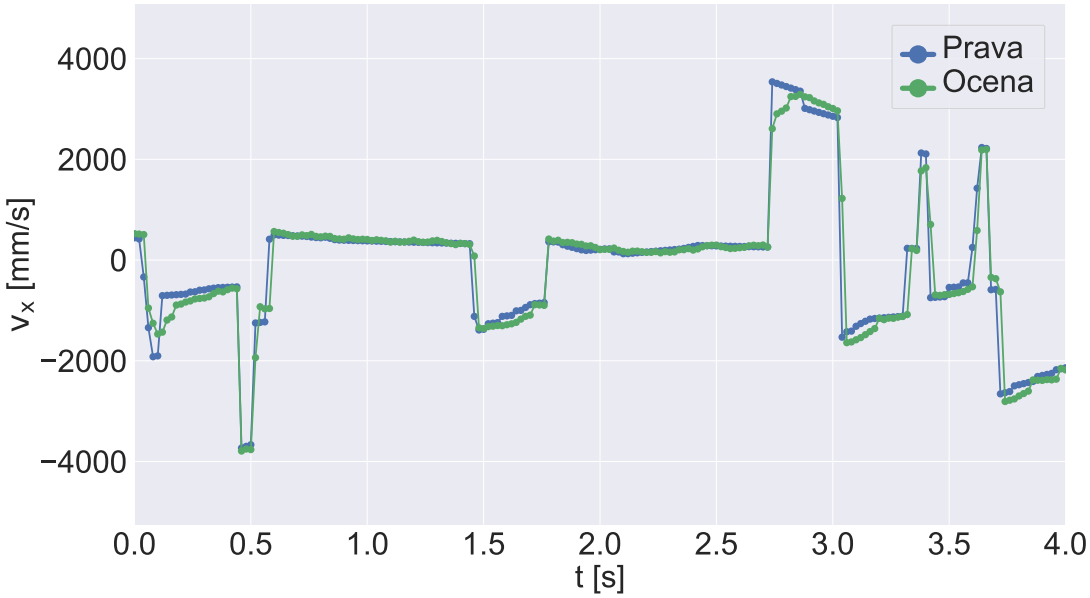
kjer $\overline{v}$ predstavlja zglajeno hitrost koordinate x oz. y, $\dot{p}$ odvod koordinate, c konstanto glajenja in σ vrednost standardnega odklona šuma. Parametri filtrov so predstavljeni s tabelo 4.12. Delovanje filtra prikazuje slika 4.13.

Tabela 4.12: Parametri filtra hitrosti v posameznem koraku glajenja

korak	c	σ [mm/s]
1.	/	/
2.	0,5	500
3.	0,5	100

4.3.3.4 Odprava vpliva zakasnitve na položaju palic

Modela iz enačbe (4.11) pri ocenjevanju položaja palic ne moremo uporabiti, saj hitrosti ne moremo obravnavati kot konstanto. Zakasnitve smo zato odpravljali

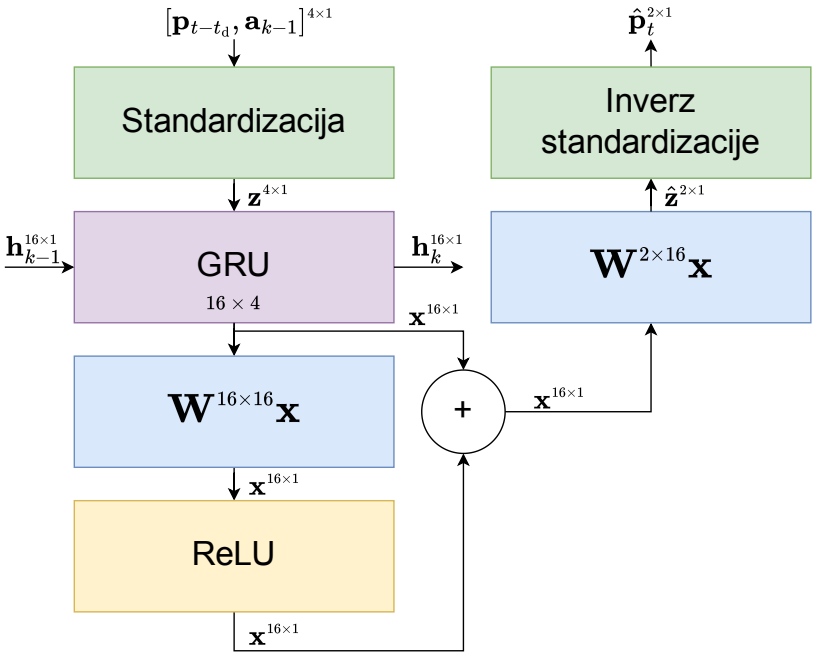


Slika 4.13: Primer ocenjevanja hitrosti žoge. Z modro barvo je prikazana prava hitrost, z zeleno pa njena ocena. Slika prikazuje hitrost po osi x.

z modelom, ki temelji na preteklih stanjih in akcijah. Model je bil izdelan oz. naučen z globokim nadzorovanim učenjem.

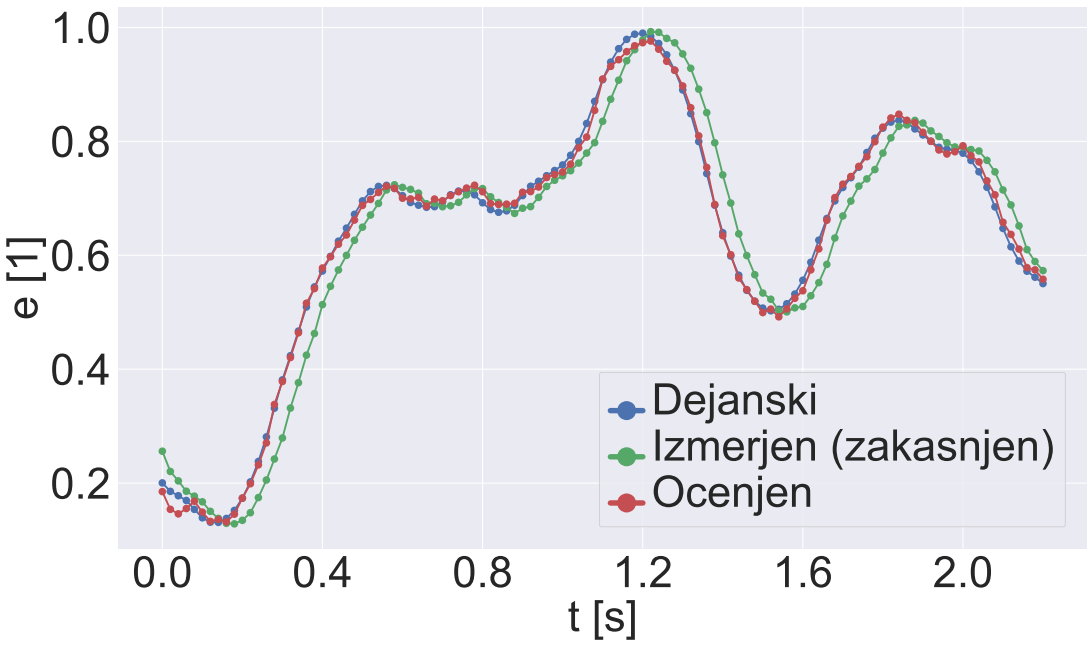
V simulaciji smo s strategijo, ki je bila naučena brez prisotnosti časovne zakasnitve, posneli 2000 epizod, kjer smo snemali zakasnjen položaj po iztegu in zasuku $\mathbf{p}_{t-t_\mathrm{d}} = [e_{t-t_\mathrm{d}}, r_{t-t_\mathrm{d}}]$, akcije $\mathbf{a}_k$ ter nezakasnjen položaj $\mathbf{p}_t$. Podatke smo nato uporabili za učenje modela, ki je imel obliko nevronske mreže. Slednja je ponazorjena s sliko 4.14. Model smo učili napovedovanja pravega položaja $\mathbf{p}_t$ na podlagi zaporedij zakasnjenih položajev $\mathbf{p}_{t-t_\mathrm{d}}$, akcij $\mathbf{a}_{k-1}$ in skritega stanja GRU $\mathbf{h}_{k-1}$, torej $\mathcal{F} : (\mathbf{p}_{t-t_\mathrm{d}}, \mathbf{a}_{k-1}, \mathbf{h}_{k-1}) \rightarrow [\mathbf{p}_t, \mathbf{h}_k]$, pri čemer se je časovna zakasnitev $t_\mathrm{d} \in [10,\ 90]$ ms naključno spreminjala med epizodami. Časovna zakasnitev je bila zvezna v času z ločljivostjo dveh milisekund. Oznaka k predstavlja časovni korak, ki ga je mogoče pretvoriti v čas z enačbo $t = k\,T_\mathrm{s}$, pri čemer je $T_\mathrm{s} = 20\,\mathrm{ms}$ vzorčna frekvenca. Za optimizacijo modela smo uporabili kriterijsko funkcijo povprečne kvadratne napake (angl. *Mean Squared Error* – MSE) med pravim in napovedanim položajem. Kriterijsko funkcijo smo minimizirali z algoritmom Adam.

Primer ocenjevanja položaja (iztega) palice prikazuje slika 4.15. Iz slike je



Slika 4.14: Arhitektura modela, ki je bil uporabljen za napoved nezakasnjenega položaja palice. Plasti, ki sta označeni z zeleno barvo, predstavljata operacijo iz enačbe (4.5) oz. njen inverz in imata obe enaki vrednosti parametrov (povprečje in standardni odklon).

razvidno, da napovedi odstopajo od pravih položajev, zato smo prvotne strategije dodatno učili v zakasnjenem okolju. Med dodatnim učenjem smo ocenjevali tako položaj palice z zgoraj opisanim modelom, kot tudi položaj žoge z enačbo (4.11).



Slika 4.15: Ocenjevanje iztega palice (e) vratarja pri zakasnitvi stanja 40 ms

5 Rezultati

Tu so podani končni rezultati vrednotenja posameznih strategij v simulaciji, zatem vrednotenje vseh strategij skupaj v pravem okolju, na koncu pa primerjave pri različnih oblikah nagrad, stanj, akcij itd. Vsem rezultatom je skupno, da je epizoda trajala največ 500 korakov (10 sekund). Rezultati, ki so podani s tabelo, so bili pridobljeni s petimi vrednotenji istega modela. V vsakem vrednotenju je bilo izvedenih 5000 epizod. Uporabljen je bil najboljši model iz množice neodvisnih poskusov, ki je bil izbran na podlagi treh ponovitev vrednotenja.

Krivulje, ki se nanašajo na vratarja in obrambo, predstavljajo povprečno vrednost desetih ponovitev, območje okoli krivulje pa negotovost.

Rezultati, ki se nanašajo na napadalca, so bili pridobljeni na podlagi enega poskusa algoritma PBT. Pri tem je znotraj tega poskusa delovalo 20 poskusov algoritma PPO. Razlog za le en poskus algoritma PBT izhaja iz dolgega časa učenja (reda nekaj dni). Vzporedno učenje z več poskusi algoritma PBT naenkrat ni bilo mogoče, saj je že en poskus zasedal skoraj vse procesorske niti.

5.1 Končni rezultati

5.1.1 Napadalec

Glavna metrika za vrednotenje napadalca predstavlja razmerje med epizodami z uspešnim strelom na gol in vsemi epizodami. Strel se kot uspešnega šteje, če je žoga pristala v golu. Zaradi lažje izvedbe se med uspešne epizode štejejo tudi tiste z več kot enim izvedenim strelom, torej vse z zadetim golom znotraj desetih sekund od začetka epizode.

Vrednotenje strategije napadalca, ki smo jo dodatno učili v zakasnjenem okolju, prikazuje tabela 5.1. Pri vključitvi časovne zakasnitve na stanje je razvidno, da uspešnost precej upade. Ob vključitvi kompenzacijskih modelov se uspešnost povrne na skoraj enako raven kot v nezakasnjenem okolju. Tabela 5.2 prikazuje vrednotenje strategije napadalca brez dodatnega učenja v zakasnjenem okolju. Razvidno je, da je model brez dodatnega učenja v zakasnjenem okolju slabši v vseh navedenih pogojih v tabeli.

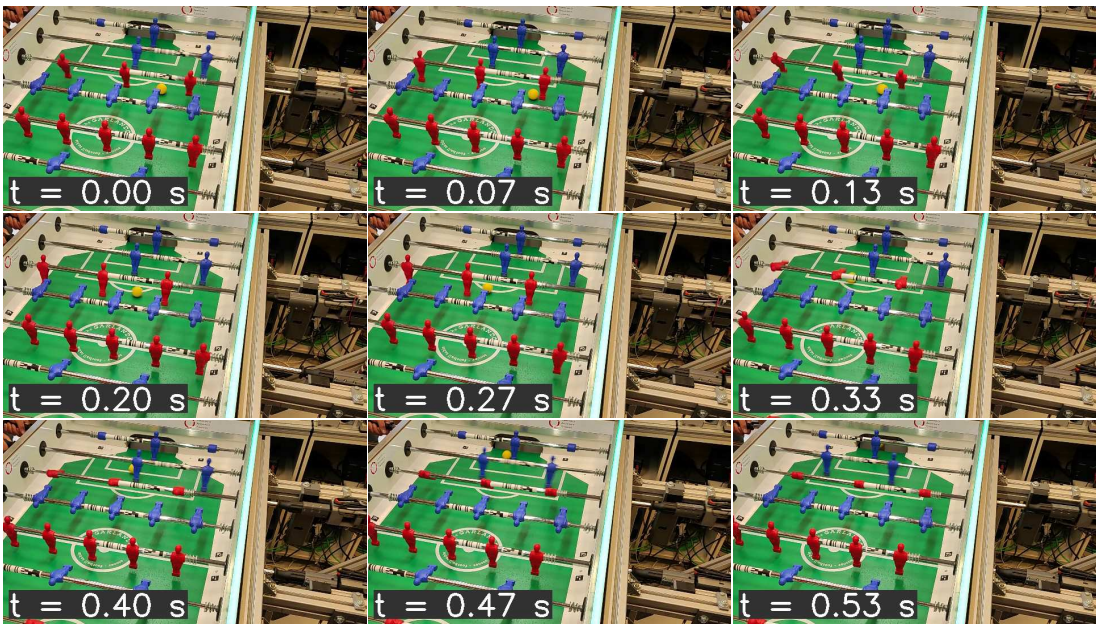
Tabela 5.1: Delež uspešnih epizod (povprečje ± negotovost) pri streljanju napadalca na gol nasprotnika s strategijo, ki smo jo **dodatno učili v zakasnjenem okolju**

Zakasnitev [ms]	Kompenzacija	**Uspešnost** [%]
0	brez	$91{,}820 \pm 0{,}150$
40	brez	$42{,}488 \pm 0{,}285$
40	žoga	$49{,}728 \pm 0{,}171$
40	žoga in palica	$88{,}464 \pm 0{,}121$

Tabela 5.2: Delež uspešnih epizod (povprečje ± negotovost) pri streljanju napadalca na gol nasprotnika s strategijo, ki smo jo **učili v nezakasnjenem okolju**

Zakasnitev [ms]	Kompenzacija	**Uspešnost** [%]
0	brez	$90{,}212 \pm 0{,}223$
40	brez	$35{,}220 \pm 0{,}288$
40	žoga	$38{,}844 \pm 0{,}381$
40	žoga in palica	$80{,}512 \pm 0{,}434$

Delovanje strategije napadalca (v pravem okolju) je prikazano na sliki 5.1, kjer agent žogo prestreže, poda soigralcu na isti palici in ustreli na gol nasprotnika.



Slika 5.1: Podaja in strel RL-agenta napadalca v gol nasprotnika v pravem okolju

5.1.2 Vratar in obramba

Glavna metrika za vrednotenje strategije vratarja in strategije obrambe je delež ubranjenih strelov, ki bi brez posredovanja vratarja ali obrambe vodili v prejem gola. Neubranjene strele zaznavamo s preverjanjem prejetih golov, medtem ko zaznavanje ubranjenih strelov poteka v več korakih. Začnemo z zaznavo trka žoge s katerimkoli od igralcev lastne ekipe. Tri časovne korake ($3 \times 20\,\mathrm{ms}$) po koncu zadnjega trka nato izračunamo napovedano koordinato y položaja žoge, kjer bi žoga brez trka pristala v golu. Prav tako izračunamo novo napovedano koordinato y položaja žoge kot posledico trka žoge in igralca. Napovedano koordinato y položaja žoge izračunamo po enačbi (5.1), kjer $[x_\mathrm{b}, y_\mathrm{b}]^\mathrm{T}$ predstavlja trenutni položaj žoge. V primeru, da je žoga pred trkom potovala proti golu ($y_b^\mathrm{g} \in [248,\ 452]$ mm in $\dot{x}_\mathrm{b} < 0\,\mathrm{mm/s}$) ter po trku ni več potovala tja, smo trk šteli kot ubranjen strel.

$$y_\mathrm{b}^\mathrm{g} = y_\mathrm{b} + \dot{y}_\mathrm{b}\,\frac{0 - x_\mathrm{b}}{\dot{x}_\mathrm{b}} \tag{5.1}$$

Rezultate vrednotenja strategij vratarja in obrambe, ki smo jih dodatno učili v zakasnjenem okolju, podaja tabela 5.3. Rezultate vrednotenja strategij vratarja in obrambe, ki smo jih učili le v nezakasnjenem okolju, podaja tabela 5.4. Izkaže se, da se je uspešnost branjenja obrambe skupaj z vratarjem le malo izboljšala z dodatnim učenjem v zakasnjenem okolju.

Izkaže se tudi, da je branjenje le vratarja v zakasnjenem okolju boljše brez dodatnega učenja. Razlog za to verjetno izhaja iz neprimernih hiperparametrov za učenje v zakasnjenem okolju, vendar tega v delu nismo raziskali. Prav tako je branjenje boljše, če strategijo obrambe učene v zakasnjenem okolju vrednotimo ob prisotnosti vratarja, ki deluje s strategijo učeno le v nezakasnjenem okolju.

Tabela 5.3: Delež uspešno ubranjenih strelov (povprečje ± negotovost) vratarja in obrambe s strategijama, ki sta bili **dodatno učeni zakasnjenem okolju**

Zakasnitev [ms]	Kompenzacija	**Uspešnost** [%]	
		Vratar	Vratar z obrambo
0	brez	89,445 ± 0,125	96,471 ± 0,086
40	brez	64,891 ± 0,229	84,897 ± 0,119
40	žoga	69,997 ± 0,044	84,978 ± 0,104
40	žoga in palica	77,476 ± 0,139	91,026 ± 0,113

Tabela 5.4: Delež uspešno ubranjenih strelov (povprečje ± negotovost) vratarja in obrambe s strategijama, ki sta bili **učeni v nezakasnjenem okolju**

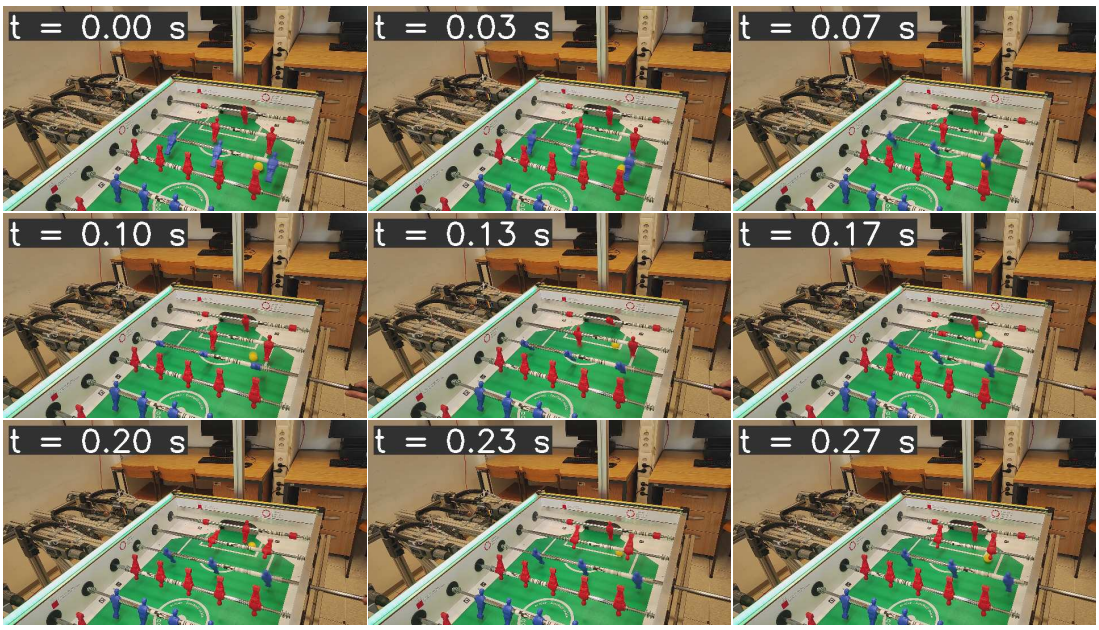
Zakasnitev [ms]	Kompenzacija	**Uspešnost** [%]	
		Vratar	Vratar z obrambo
0	brez	91,148 ± 0,090	96,891 ± 0,074
40	brez	64,493 ± 0,186	85,082 ± 0,127
40	žoga	69,014 ± 0,110	83,208 ± 0,185
40	žoga in palica	78,635 ± 0,127	90,713 ± 0,101

Delovanje strategij vratarja in obrambe (v pravem okolju) je prikazano na sliki 5.2, kjer agenta prestrezata strele nasprotnika. Iz slike je razvidno, da ima na začetku nadzor nad žogo nasprotnikov napadalec. Agenta sta premaknjena proti položaju žoge, vendar vseeno stran od mejnih vrednosti iztega, s čimer pokrivata

Tabela 5.5: Delež uspešno ubranjenih strelov (povprečje ± negotovost) vratarja in obrambe, strategij. Strategija **vratarja** je bila učena v **nezakasnjenem okolju**, medtem ko je bila strategija **obrambe dodatno učena v zakasnjenem okolju**.

Zakasnitev [ms]	Kompenzacija	**Uspešnost** [%]
		Vratar z obrambo
0	brez	97,066 ± 0,064
40	brez	84,574 ± 0,179
40	žoga	84,943 ± 0,159
40	žoga in palica	91,275 ± 0,089

čim večji del gola. Nasprotnikov napadalec nato žogo ustreli tako, da se ta začne proti golu gibati pod neničelnim kotom. Žogo prestreže agent vratar, ki se je le malo izmaknil iz začetnega položaja.



Slika 5.2: Branjenje gola s strani RL-agentov vratarja in obrambe v pravem okolju

5.1.3 Igra proti človeku

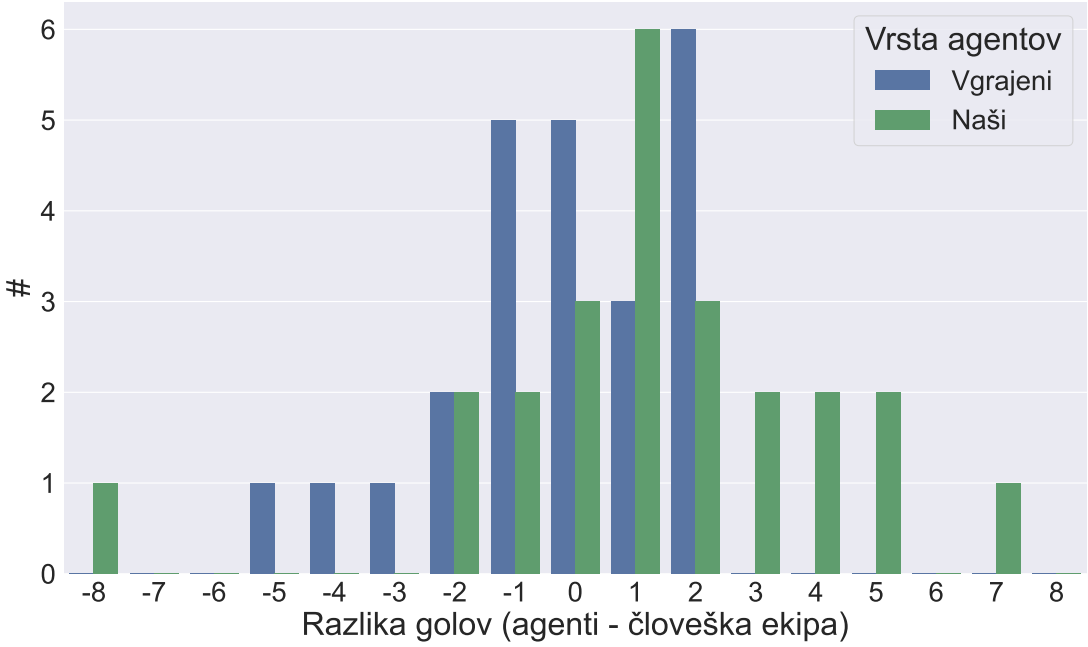
Pomemben del našega dela je tudi prenos v pravo okolje, kjer agenti delujejo na pravi mizi namiznega nogometa proti nasprotni človeški ekipi. Zaradi tehničnih

omejitev enake inicializacije okolja, kot je ta bila v simulaciji, nismo mogli izvesti. Prav tako nismo mogli imeti enakega nasprotnika, saj na sistemu FuzbAI nasprotnikova stran nima motorjev in je namenjena človeškim igralcem. Prenos v pravo okolje smo zato vrednotili z več igrami proti več nasprotnikom, pri čemer smo vse agente vrednotili kot celoto.

Proti agentom se je pomerilo 12 ekip, pri čemer je vsaka človeška ekipa igrala 4 igre. Dve igri sta potekali proti agentom izdelanih v tem delu. To so vsi RL-agenti in tudi FSM-agent, ki je upravljal palico srednje vrste. Preostali 2 igri sta potekali proti agentom, ki so vgrajeni v glavni računalnik na mizi FuzbAI. Slednjih nismo razvili mi. Prav tako sta človeška igralca med zaporednima igrama proti istemu agentu zamenjala vlogi. Tako sta oba človeška igralca igrala v obrambni in napadalni vlogi. Skupno je bilo odigranih 24 iger proti posamezni skupini agentov oz. 48 iger skupno proti obema skupinama agentov. Vrstni red agentov (vgrajeni agenti ali naši agenti) je bil pred prvo igro določen naključno. Proti izžrebanim agentom se je človeški par najprej eno minuto ogreval. Nato je ob istih pogojih pričel prvo igro. Vsaka igra je potekala dve minuti.

Rezultate vrednotenja proti človeškim ekipam prikazuje slika 5.3. Prikazani so rezultati tako vgrajenih agentov in agentov, ki smo jih izdelali v tem delu, vendar popolna primerjava ni povsem smiselna, saj so agenti delovali v različnih pogojih. Agenti, ki smo jih izdelali v tem delu, so bili vodeni preko prenosnega računalnika in protokola Ethernet. Vgrajeni agenti so bili vodeni preko glavnega računalnika in tako niso potrebovali Ethernet povezave. Dodatno so vgrajeni agenti spreminjali svojo težavnost glede na število prejetih oz. zadetih golov. Rezultate obeh vrst agentov smo vključili predvsem za podajo občutka pričakovanih vrednosti.

Iz zelenega dela grafa na sliki 5.3 je razvidno, da je bilo največ iger uravnoteženih, saj so agenti, ki smo jih izdelali v tem delu, ob koncu igre največkrat vodili le za eno točko. Nekaj iger je sicer bilo izgubljenih, vendar se graf vseeno nagiba proti desni, kar pomeni več zadetih golov kot prejetih, zato menimo, da je prenos v pravo okolje uspel.



Slika 5.3: Histogram razlik med zadetimi in prejetimi goli pri vrednotenju agentov proti človeškim igralcem. Modra barva označuje agente, ki so vgrajeni v mizo, zelena pa agente, ki smo jih izdelali v tem delu. Vgrajeni agenti so med igro spreminjali svojo težavnost glede na rezultat, zato primerjava služi zgolj kot orientacija in tako podaja občutek območja števil.

5.2 Primerjalne študije

V tem poglavju so podani rezultati primerjalnih študij, kjer smo spreminjali nagrado, nasprotnika ipd.

Nekatere študije so bile izvedene ob različnih časih in tako ob različnih nastavitvah okolja, začetnih pogojih, fizikalnih parametrih itd., zato neposredna primerjava s končnimi rezultati v 5.1 ni primerna, niti ni primerna primerjava z drugimi primerjalnimi študijami.

5.2.1 Napadalec

5.2.1.1 Učenje in vrednotenje z različnimi nasprotniki

Učenje končne strategije napadalca je potekalo v celoti brez prisotnosti nasprotnika. Razlog za to izhaja iz primerjalne študije, kjer smo preverjali vpliv nasprotnika na uspešnost zaporedja podaja in strel. Neodvisno smo učili več strategij napadalca. Posamezna strategija je med učenjem imela različno konfiguracijo nasprotnika:

1. brez prisotnosti nasprotnika,
2. prisotnost RL-vratarja iz starejših poskusov in
3. prisotnost vratarja izbranega iz množice: {FSM-vratar, sinusno gibanje, RL-vratar iz starejših poskusov}.

Po učenju smo naučene strategije vrednotili na podlagi uspešnosti zaporedja podaje in strela, pri čemer je vrednotenje izvedeno pri različnih konfiguracijah nasprotnika. Med poskusi se je poleg nasprotnika razlikoval le prostor stanj. Slednjemu smo pri učenju proti množici vrst agentov (konfiguracija 3) k obstoječemu stanju (opisanemu v poglavju 4.3.1.3), dodali še izteg nasprotnikove palice. Uspešnost podaja tabela 5.6. Rezultati pripadajo najboljšemu modelu izmed dvajsetih poskusov algoritma PPO, znotraj enega poskusa algoritma PBT.

Tabela 5.6: Delež epizod z uspešno podajo in strelom (povprečje ± negotovost [%]) pri zakasnitvi 40 ms, kompenzaciji zakasnitve žoge in lastni kompenzaciji zakasnitve palice. Indeks vrstice določa konfiguracijo nasprotnika med učenjem in posledično določa tudi strategijo. Indeks stolpca določa konfiguracijo nasprotnika med vrednotenjem.

Konfiguracija nasprotnika		**Uspešnost** [%]	
		Vrednotenje	
		RL-vratar (2)	Množica (3)
Učenje	Brez nasprotnika (1)	**65,300 ± 0,183**	**70,724 ± 0,120**
	RL vratar (2)	62,384 ± 0,270	52,220 ± 0,338
	Množica (3)	56,648 ± 0,220	56,352 ± 0,571

5.2.1.2 Učenje z zahtevano predhodno podajo

Funkcija nagrade končne strategije napadalca (enačba (4.8)) dovoljuje takojšen strel s sredinskim igralcem, stranska igralca pa morata žogo podati sredinskemu igralcu. V tej študiji primerjamo končni rezultat najboljšega modela ob učenju z dvema različnima funkcijama nagrade:

1. z dovoljenim takojšnjim sredinskim strelom, kot opisano v enačbi (4.8), z vrednostjo uteži 0,005 namesto 0,010 pred členom, ki vsebuje sile (utež je bila uravnavana tudi po zaključku te primerjalne študije)
2. in zahtevano podajo ne glede na igralca (dodatek P) : $R^{(a)}_{\text{shot}} = R^{(a)}_{\text{pass}} + P\,M\,(2\,S + 0{,}005\,(f_x + \min(\max(f_z, -f_x), 0)))$, kjer $P \in \{0,\ 1\}$ predstavlja indikator o vsaj eni predhodno izvedeni podaji.

Tabela 5.7: Delež epizod z uspešno zadetim golom (povprečje ± negotovost [%]) pri zakasnitvi 40 ms, kompenzaciji zakasnitve položaja žoge in lastni kompenzaciji zakasnitve položaja palice

Brez zahtevane podaje (1)	Z zahtevano predhodno podajo (2)
87,984 ± 0,123	82,756 ± 0,181

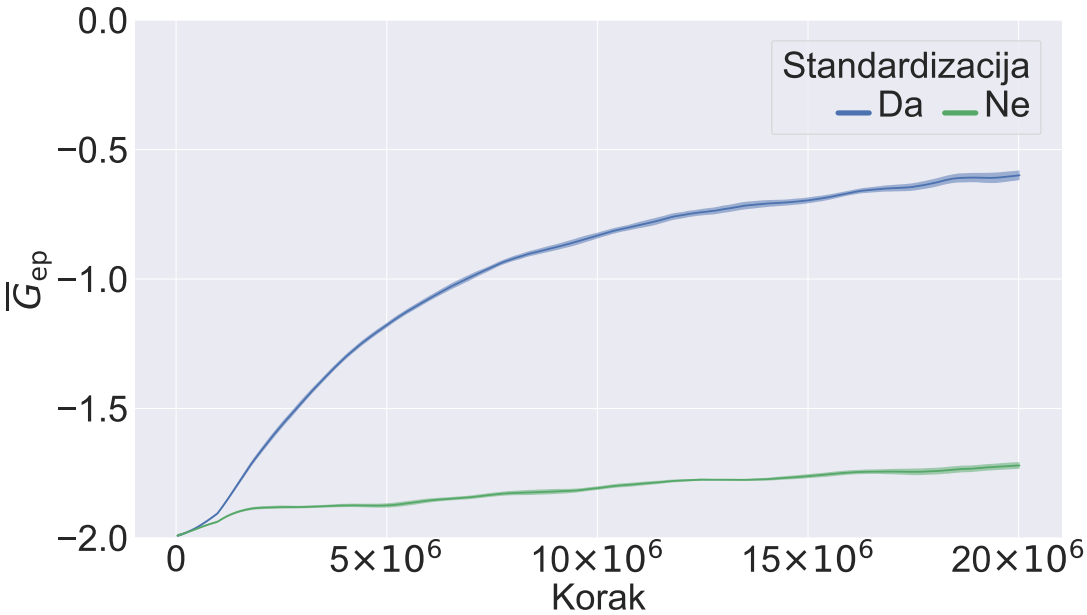
5.2.2 Vratar in obramba

Primerjalne študije vratarja in obrambe prikazujejo rezultate učenja v prisotnosti FSM-nasprotnika. Kljub temu, da sta končna modela učena tudi ob prisotnosti RL-nasprotnika napadalca, slednjega v primerjalnih študijah nismo vključevali zaradi povečane računske zahtevnosti. Preostala konfiguracija je enaka kot opisano v poglavju 4.3.2, z izjemo sprememb opisanih pri posamezni primerjalni študiji.

5.2.2.1 Standardizacija stanja

Standardizacija vhoda (enačba (4.5)) nevronskim mrežam omogoča lažjo optimizacijo in pogosto vodi do boljših rešitev tudi v spodbujevalnem učenju. Pri-

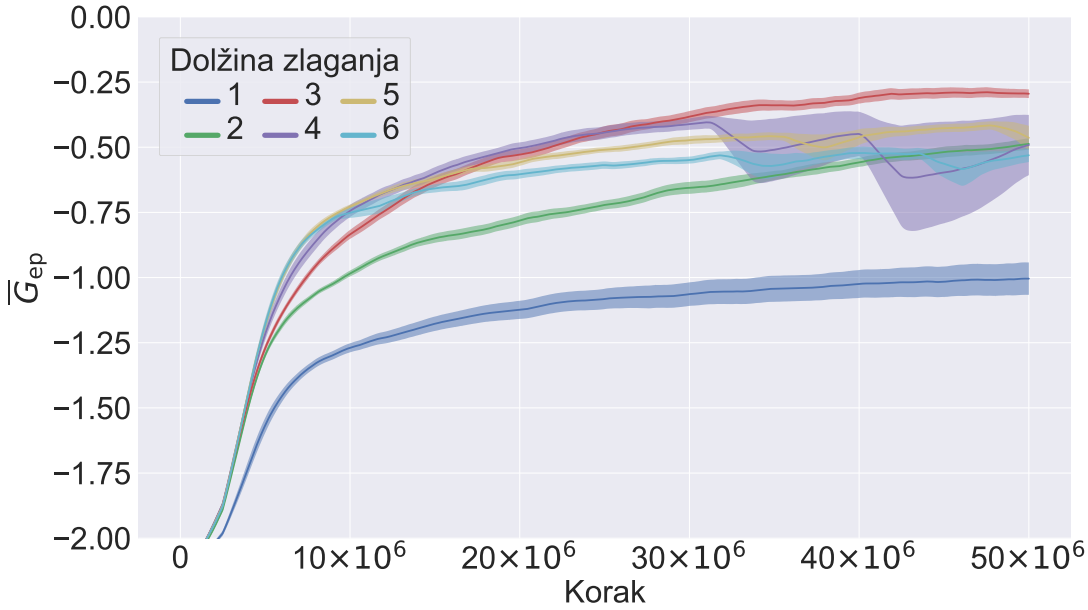
merjavo brez in s standardizacijo smo izvedli le za strategijo vratarja in sklepali, da pomaga tudi strategijam za vodenje preostalih palic. Vpliv standardizacije na donos med učenjem prikazuje slika 5.4. Z izjemo standardizacije sta bili obe krivulji pridobljeni v skladu s poglavjem 4.3.2, zato vrednost epizodnega donosa $\overline{G}_{\text{ep}} = -2$ približno ustreza ničelni uspešnosti branjenja gola, vrednost $\overline{G}_{\text{ep}} = 0$ pa približno ustreza popolnemu branjenju.



Slika 5.4: Vpliv standardizacije stanja na povprečni epizodni donos ($\overline{G}_{\text{ep}}$) pri učenju strategije vratarja

5.2.2.2 Dolžnina zlaganja stanj

V študiji dolžine zlaganja stanj smo preverjali kakšen vpliv na donos ima število preteklih osnovnih stanj oz. meritev, ki so združene v skupno razširjeno stanje. Preizkusili smo zlaganje enega, dveh, treh, štirih, petih in šestih preteklih osnovnih stanj. Primerjavo donosa med učenjem prikazuje slika 5.5. Iz slike je razvidno, da je najmanjša dolžina zlaganja, kjer še vedno dobimo relativno najboljše rezultate, enaka 3. Učenje na začetku kaže višji donos pri poskusih, ki stanje zlagajo z dolžino 4, 5 ali 6. Slednji so proti koncu kazali bolj pogosto nestabilnost učenja, zato je tam njihovo povprečje nižje, negotovost pa višja.



Slika 5.5: Povprečni epizodni donos ($\overline{G}_{\text{ep}}$) med učenjem strategije vratarja pri različnih dolžinah zlaganja osnovnih stanj. Dolžina zlaganja 1 pomeni vključitev le najnovejšega osnovnega stanja.

6 Zaključek

Med izdelavo inteligentnih agentov s spodbujevalnim učenjem smo se naučili, da je le-to precej težje kot se na prvi pogled zdi. Prvotna misel, „agenti, ki se bodo sami naučili igranja", se je v praksi izkazala za precej težji problem kot le pritisk gumba. Dodatno smo si zastavili precej težji problem kot sorodna dela, in sicer smo se odločili za izdelavo sistema več agentov skupaj, ki morajo delovati v pravem okolju, tudi proti človeškemu nasprotniku.

Z rezultati našega dela smo zadovoljni, saj so agenti sposobni delovanja tako v simulaciji kot tudi v pravem okolju.

Delo v prihodnosti dopušča predvsem izboljšavo agenta napadalca. Ta žogo dobro prestreza ob nespremenjeni smeri gibanja žoge, kar pa je mogoče le brez nasprotnika. Delo v prihodnosti tako vključuje predvsem izboljšanje obrambne drže agenta napadalca.

Literatura

[1] A. Radford, K. Narasimhan, T. Salimans in S. Ilya, "Improving language understanding by generative pre-training." https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf, 2018. Dostopano: 6. junij 2025.

[2] Google Cloud, "What is artificial intelligence (AI)?." https://cloud.google.com/learn/what-is-artificial-intelligence. Dostopano: 5. junij 2025.

[3] ScienceDirect, "Expert system." https://www.sciencedirect.com/topics/biochemistry-genetics-and-molecular-biology/expert-system. Dostopano: 17. avgust 2025.

[4] S. De Blasi, S. Klöser, A. Müller, R. Reuben, F. Sturm in T. Zerrer, "Kicker: An industrial drive and control foosball system automated with deep reinforcement learning," *Journal of Intelligent & Robotic Systems*, vol. 102, no. 1, str. 20, 2021.

[5] T. Rohrer, L. Samuel, A. Gashi, G. Grieser in E. Hergenröther, "Foosball table goalkeeper automation using reinforcement learning.," v *LWDA*, str. 173–182, 2021.

[6] A. Gashi, E. Hergenröther in G. Grieser, "Efficient training of foosball agents using multi-agent competition," v *Science and Information Conference*, str. 472–492, Springer, 2023.

[7] J. Croenen, J. Bürger in W. Meert, "Reinforcement learning of action sequences in table football," v *Proceedings of BNAIC/BeNeLearn 2024*, Utrecht University, 2024.

[8] M. Bošnak in G. Klančar, "Fast and Reliable Alternative to Encoder-Based Measurements of Multiple 2-DOF Rotary-Linear Transformable Objects Using a Network of Image Sensors with Application to Table Football," *Sensors*, vol. 20, no. 12, 2020.

[9] G. James, D. Witten, T. Hastie, R. Tibshirani in J. Taylor, *An introduction to statistical learning: With applications in Python*. Springer, 2023.

[10] M. Minsky in S. Papert, "An introduction to computational geometry," *Cambridge tiass., HIT*, vol. 479, no. 480, str. 104, 1969.

[11] Stanford, "CS231n: Introduction to RNN." `https://cs231n.github.io/rnn/`. Dostopano: 6. junij 2025.

[12] S. Hochreiter in J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, str. 1735–1780, 11 1997.

[13] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk in Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," v *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (A. Moschitti, B. Pang in W. Daelemans, ur.), (Doha, Qatar), str. 1724–1734, Association for Computational Linguistics, okt. 2014.

[14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser in I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[15] D. E. Rumelhart, G. E. Hinton in R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, str. 533–536, 1986.

[16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai in S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," v *Advances in Neural Information Processing Systems* (H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox in R. Garnett, ur.), vol. 32, Curran Associates, Inc., 2019.

[17] D. P. Kingma in J. Ba, "Adam: A method for stochastic optimization," v *International Conference on Learning Representations (ICLR)*, 2015.

[18] OpenAI, "Spinning up in deep reinforcement learning: Introduction to RL." `https://spinningup.openai.com/en/latest/spinningup/rl_intro.html`, 2018. Dostopano: 6. junij 2025.

[19] R. S. Sutton, A. G. Barto *et al.*, *Reinforcement learning: An introduction*, vol. 1. MIT press Cambridge, 1998.

[20] B. Baker, I. Akkaya, P. Zhokov, J. Huizinga, J. Tang, A. Ecoffet, B. Houghton, R. Sampedro in J. Clune, "Video pretraining (VPT): Learning to act by watching unlabeled online videos," *Advances in Neural Information Processing Systems*, vol. 35, str. 24639–24654, 2022.

[21] C. J. Watkins in P. Dayan, "Q-learning," *Machine learning*, vol. 8, str. 279–292, 1992.

[22] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, str. 529–533, 2015.

[23] G. A. Rummery in M. Niranjan, *On-line Q-learning using connectionist systems*, vol. 37. University of Cambridge, Department of Engineering Cambridge, UK, 1994.

[24] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine learning*, vol. 8, str. 229–256, 1992.

[25] T. Haarnoja, A. Zhou, P. Abbeel in S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," v *International conference on machine learning*, str. 1861–1870, Pmlr, 2018.

[26] V. Konda in J. Tsitsiklis, "Actor-critic algorithms," *Advances in neural information processing systems*, vol. 12, 1999.

[27] J. Schulman, P. Moritz, S. Levine, M. Jordan in P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," v *International Conference on Learning Representations (ICLR)*, 2016.

[28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford in O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[29] S. Kullback in R. A. Leibler, "On information and sufficiency," *The annals of mathematical statistics*, vol. 22, no. 1, str. 79–86, 1951.

[30] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. E. Gonzalez, M. I. Jordan in I. Stoica, "RLlib: Abstractions for distributed reinforcement learning," v *International Conference on Machine Learning (ICML)*, 2018.

[31] OpenAI, "Spinning Up in Deep Reinforcement Learning: Proximal Policy Optimization." `https://spinningup.openai.com/en/latest/algorithms/ppo.html`, 2018. Dostopano: 16. junij 2025.

[32] D. P. Kingma in M. Welling, "Auto-encoding variational bayes," v *International Conference on Learning Representations (ICLR)*, 2014.

[33] FE Laboratorij za avtomatiko in kibernetiko, "FuzbAISim." `https://github.com/FE-LAK/FuzbAISim`, 2024. Dostopano: 17. junij 2025.

[34] E. Coumans in Y. Bai, "PyBullet, a Python module for physics simulation for games, robotics and machine learning." `http://pybullet.org`, 2016–2021. Dostopano: 6. oktober 2025.

[35] E. Todorov, T. Erez in Y. Tassa, "MuJoCo: A physics engine for model-based control," v *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, str. 5026–5033, IEEE, 2012.

[36] G. Van Rossum in F. L. Drake Jr, *Python tutorial*, vol. 620. Centrum voor Wiskunde en Informatica Amsterdam, The Netherlands, 1995.

[37] N. D. Matsakis in F. S. Klock, "The rust language," v *Proceedings of the 2014 ACM SIGAda annual conference on High integrity language technology*, str. 103–104, 2014.

[38] PyO3 project *et al.*, "PyO3." `https://github.com/PyO3/pyo3`. Dostopano: 6. oktober 2025.

[39] PyO3 project *et al.*, "Maturin." `https://github.com/PyO3/maturin`. Dostopano: 6. oktober 2025.

[40] E. Todorov, "Convex and analytically-invertible dynamics with contacts and constraints: Theory and implementation in MuJoCo," v *2014 IEEE International Conference on Robotics and Automation (ICRA)*, str. 6054–6061, 2014.

[41] N. Hansen, "The CMA evolution strategy: a comparing review," *Towards a new evolutionary computation: Advances in the estimation of distribution algorithms*, str. 75–102, 2006.

[42] "Numerical integration." `https://mujoco.readthedocs.io/en/3.3.5/computation/#numerical-integration`. Dostopano: 15. avgust 2025.

[43] DeepMind Technologies Limited, "MuJoCo Documentation: Simulation/Model Changes." `https://mujoco.readthedocs.io/en/3.3.5/programming/simulation.html#model-changes`, 2025. Dostopano: 29. junij 2025.

[44] Y. Bengio, J. Louradour, R. Collobert in J. Weston, "Curriculum learning," v *Proceedings of the 26th annual international conference on machine learning*, str. 41–48, 2009.

[45] B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, str. 419–420, 1962.

[46] M. Jaderberg, V. Dalibard, S. Osindero, W. M. Czarnecki, J. Donahue, A. Razavi, O. Vinyals, T. Green, I. Dunning, K. Simonyan *et al.*, "Population based training of neural networks," *arXiv preprint arXiv:1711.09846*, 2017.

[47] J. Bergstra, R. Bardenet, Y. Bengio in B. Kégl, "Algorithms for hyper-parameter optimization," v *Advances in Neural Information Processing Systems* (J. Shawe-Taylor, R. Zemel, P. Bartlett, F. Pereira in K. Weinberger, ur.), vol. 24, Curran Associates, Inc., 2011.

[48] T. Akiba, S. Sano, T. Yanase, T. Ohta in M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," v *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

[49] E. Altman in P. Nain, “Closed-loop control with delayed information,” *ACM sigmetrics performance evaluation review*, vol. 20, no. 1, str. 193–204, 1992.

Dodatek

A

Izpeljava enačbe za strel FSM-agenta na gol

Izpeljava enačbe (4.4) temelji na predpostavki, da kot med točko trka na žogi in središčem žoge neposredno določa pot žoge po trku. Potreben zamik Δy_b v enačbi je grafično ponazorjen s sliko A.1, njegova izpeljava, torej enačbe (4.4), pa je sledeča:

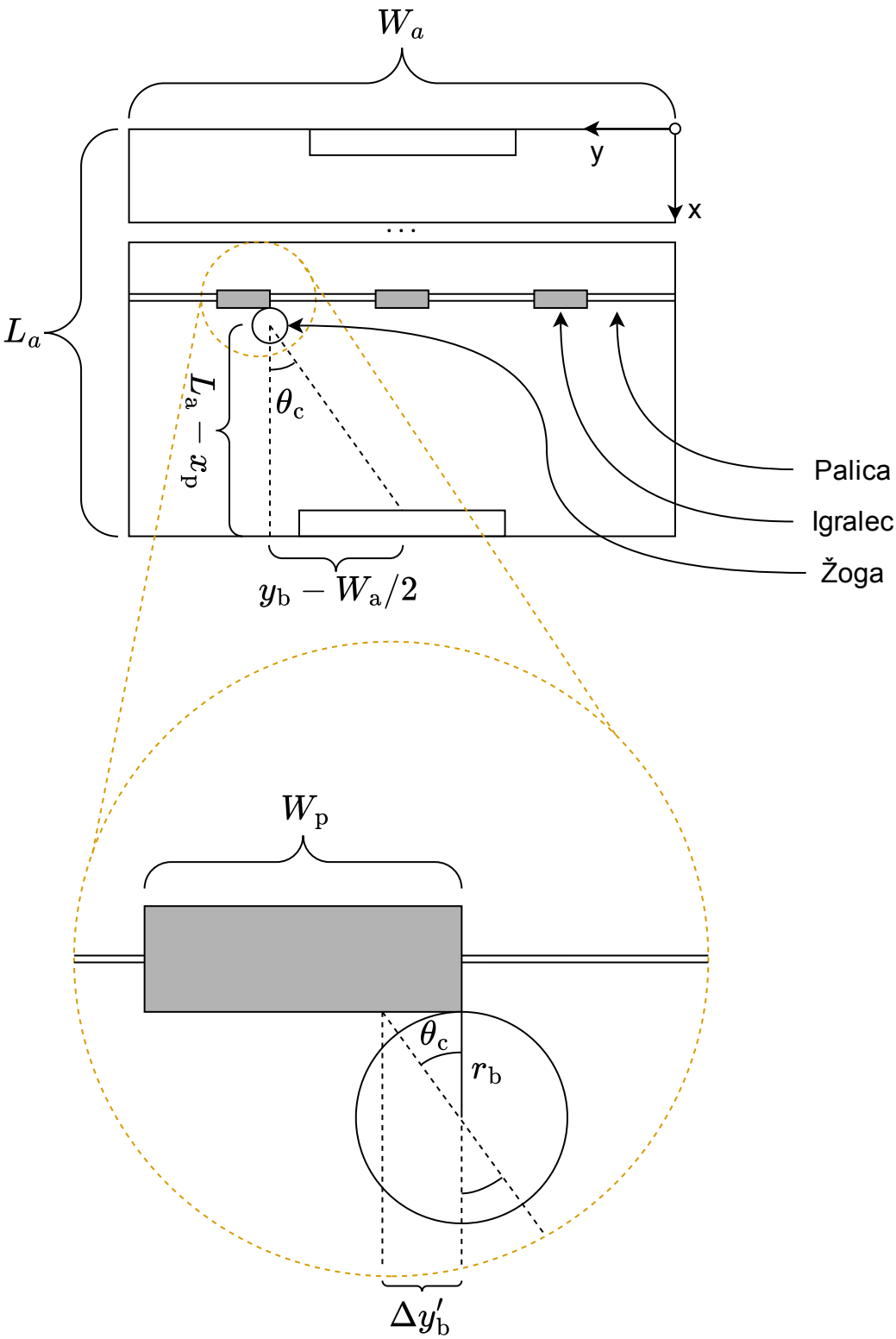
$$\tan(\theta_\mathrm{c}) = \frac{y_\mathrm{b} - W_\mathrm{a}/2}{L_\mathrm{a} - x_\mathrm{p}} = \frac{\Delta y'_\mathrm{p}}{r_\mathrm{b}}$$
$$\Rightarrow \Delta y'_\mathrm{p} = \frac{y_\mathrm{b} - W_\mathrm{a}/2}{L_\mathrm{a} - x_\mathrm{p}} \cdot r_\mathrm{b},$$

kjer $\Delta y'_\mathrm{p}$ predstavlja potreben zamik iztega palice v primeru trka z ogliščem igralca, θ_c kot trka glede na os x, W_a širino igrišča, L_a dolžino igrišča, r_b polmer žoge, x_p koordinato osi x položaja igralca in y_b koordinato y položaja žoge. Trk s prednjo površino igralca bi povzročil točkasti trk s $\theta_\mathrm{c} = 0\,\mathrm{rad}$ glede na os x, zato moramo, za potreben zamik od središča palice (Δy_p), zamiku $\Delta y'_\mathrm{p}$ dodati še razdaljo do oglišča v smeri $\Delta y'_\mathrm{p}$:

$$\Delta y_\mathrm{p} = \Delta y'_\mathrm{p} + \mathrm{sgn}(\Delta y'_\mathrm{b})\, W_\mathrm{p}/2$$

kjer W_p predstavlja širino igralca (po osi y). Izpeljana enačba za Δy_p v praksi ni delovala preveč zanesljivo, zato smo prištevanje $W_\mathrm{p}/2$ zamenjali z množenjem empirično določene konstante $k = 2$ in omejili vrednost Δy_p na $\pm 20\,\mathrm{mm}$, kot je zapisano v enačbi (4.4). Razlog za nezanesljivost izpeljane enačbe leži predvsem v predpostavki neposredne preslikave kota trka v smer gibanja. V primeru žoge

to ne velja, saj se ta lahko kotali, vrti na mestu in drsi po igralni površini. Prav tako se žoga med igro premika in tako vsebuje nezanemarljivo količino kinetične energije, kar še dodatno spodkopava izpeljano enačbo.



Slika A.1: Grafični prikaz potrebnega zamika iztega, relativno na oglišče igralca, za strel v sredino gola